Machine Intelligence 1

3.2 Bayesian Networks

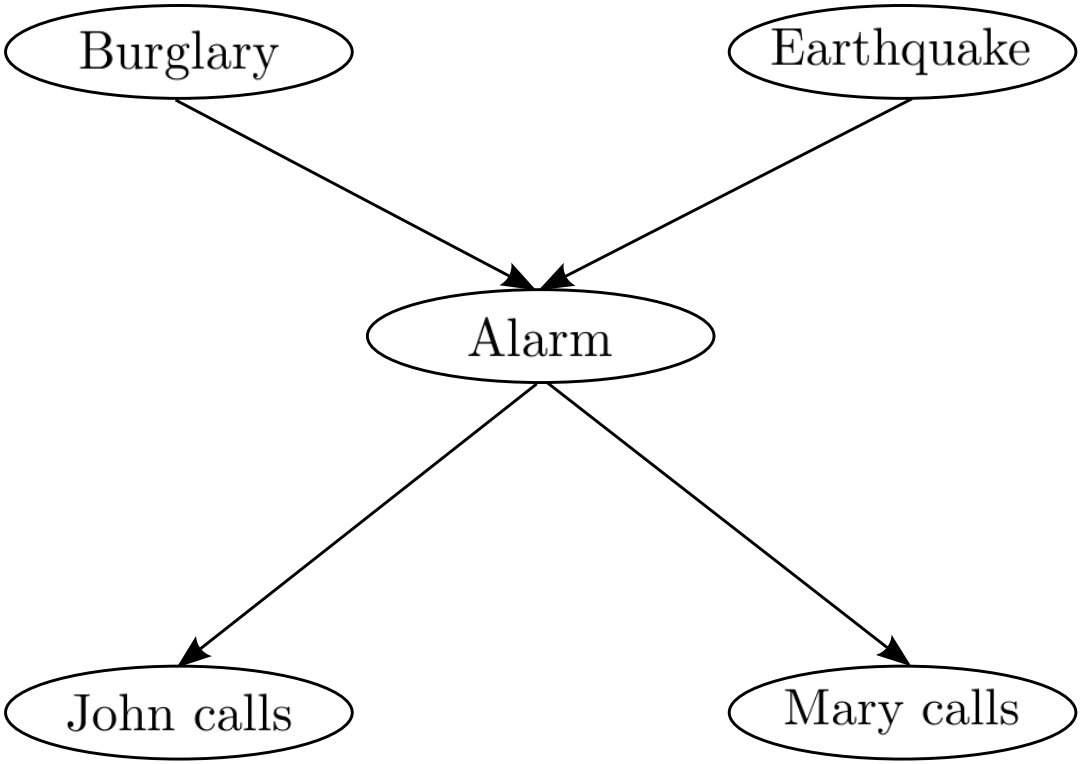
Prof. Dr. Klaus Obermayer

Fachgebiet Neuronale Informationsverarbeitung (NI)

WS 25/26

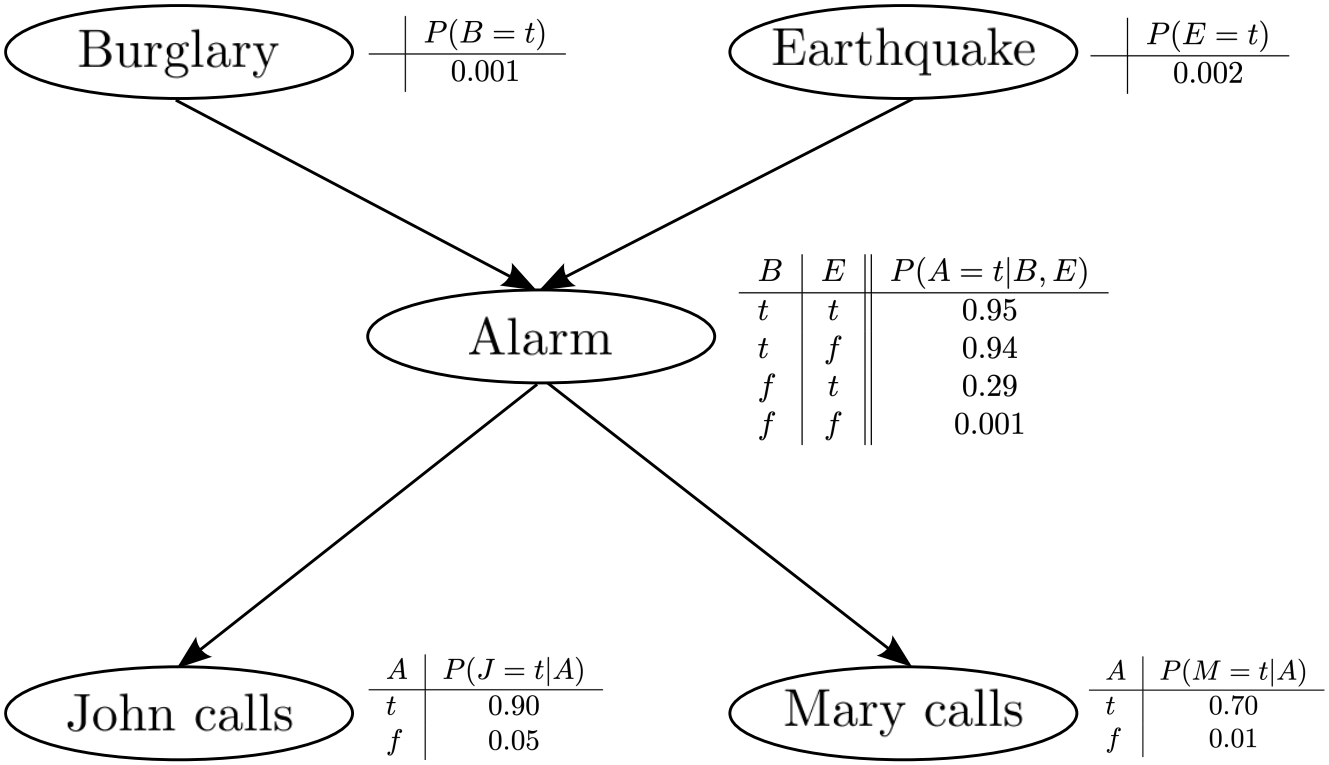
3.2.1 Directed Acyclic Graphs

A "Californian" example



$$
\begin{aligned}
P(J, M, A, B, E) &= P(J|M, A, B, E)\, P(M|A, B, E)\, P(A|B, E)\, P(B|E)\, P(E) \\
&= P(J|A)\, P(M|A)\, P(A|B, E)\, P(B)\, P(E)
\end{aligned}
$$

A "Californian" example



$P(B=t)$
0.001

$P(E=t)$
0.002

B	E	$P(A=t\|B,E)$
t	t	0.95
t	f	0.94
f	t	0.29
f	f	0.001

A	$P(J=t\|A)$
t	0.90
f	0.05

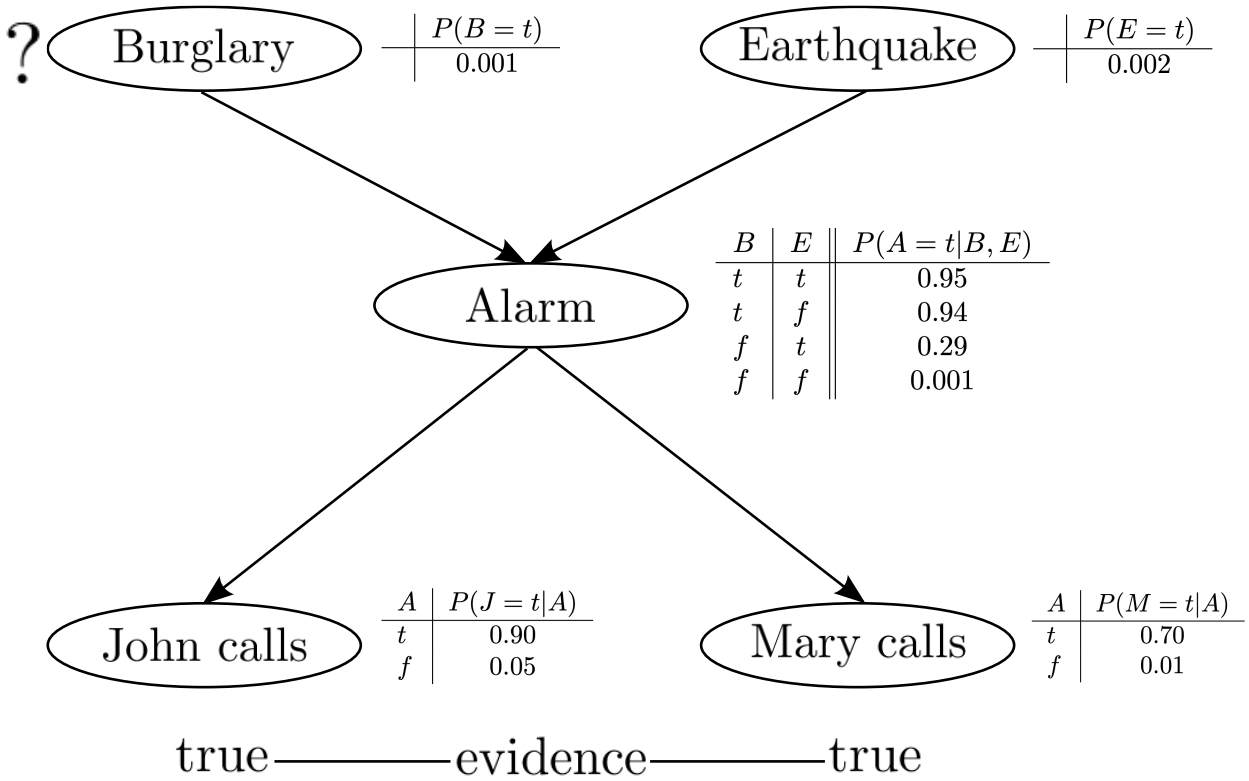
A	$P(M=t\|A)$
t	0.70
f	0.01

Considering statistical dependencies, only 10 instead of 31 entries have to be stored.

$$\begin{aligned} P(J,M,A,B,E) &= P(J|M,A,B,E)\,P(M|A,B,E)\,P(A|B,E)\,P(B|E)\,P(E) \\ &= P(J|A)\,P(M|A)\,P(A|B,E)\,P(B)\,P(E) \end{aligned}$$

Inference

- Both Mary and John are calling



$P(B=t)$
0.001

$P(E=t)$
0.002

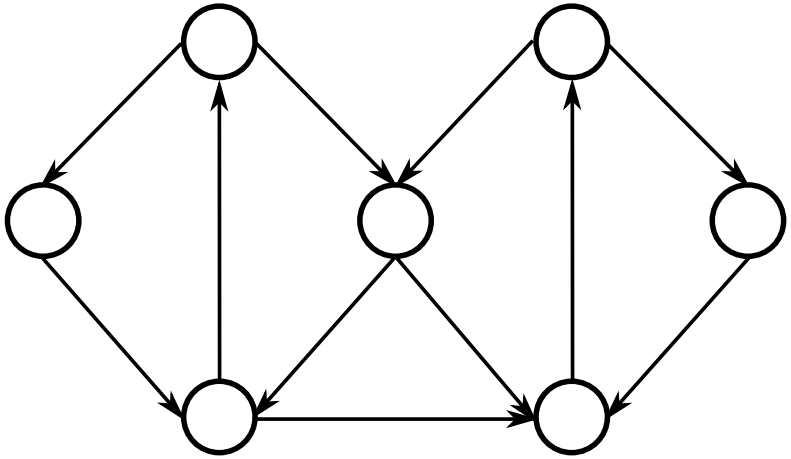
B	E	$P(A=t\|B,E)$
t	t	0.95
t	f	0.94
f	t	0.29
f	f	0.001

A	$P(J=t\|A)$
t	0.90
f	0.05

A	$P(M=t\|A)$
t	0.70
f	0.01

- Was there a burglary? (see blackboard)

Directed graphs

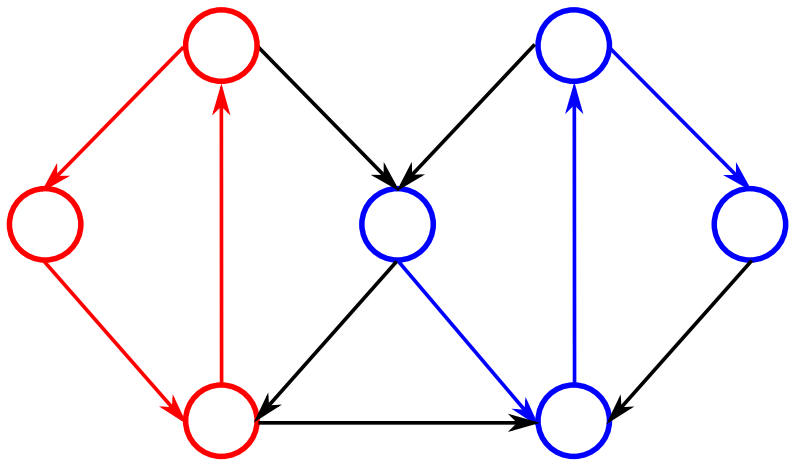


Directed graph $G = (V, K)$

$V \rightarrow$ set of nodes

$K \rightarrow$ set of directed edges

Directed graphs



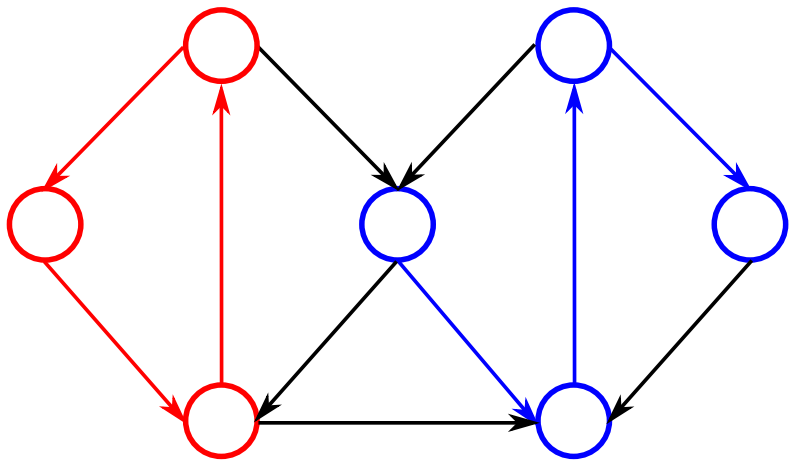
Directed graph $G = (V, K)$

$V \rightarrow$ set of nodes

$K \rightarrow$ set of directed edges

- **path:** sequence $\{x_i \in V\}_{i=1}^n$ with $(x_i, x_{i+1}) \in K$
- **cycle:** path with $x_1 = x_{n+1}$,

Directed graphs

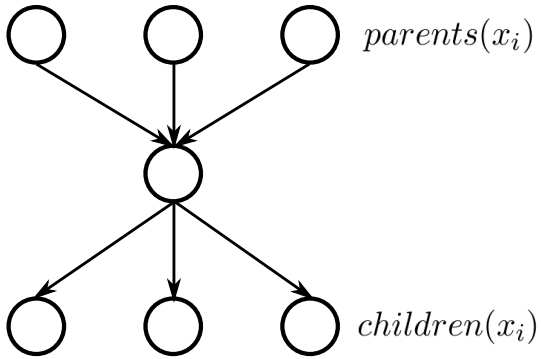


Directed graph $G = (V, K)$

$V \rightarrow$ set of nodes

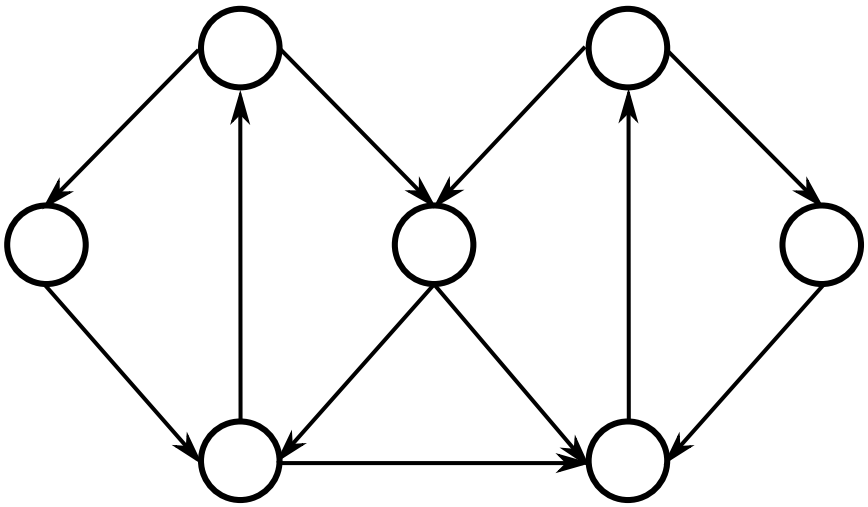
$K \rightarrow$ set of directed edges

- **path:** sequence $\{x_i \in V\}_{i=1}^{n}$ with $(x_i, x_{i+1}) \in K$
- **cycle:** path with $x_1 = x_{n+1}$,
- **parents** of x_i: $\{x_j \mid (x_j, x_i) \in K\}$
- **children** of x_i: $\{x_j \mid (x_i, x_j) \in K\}$

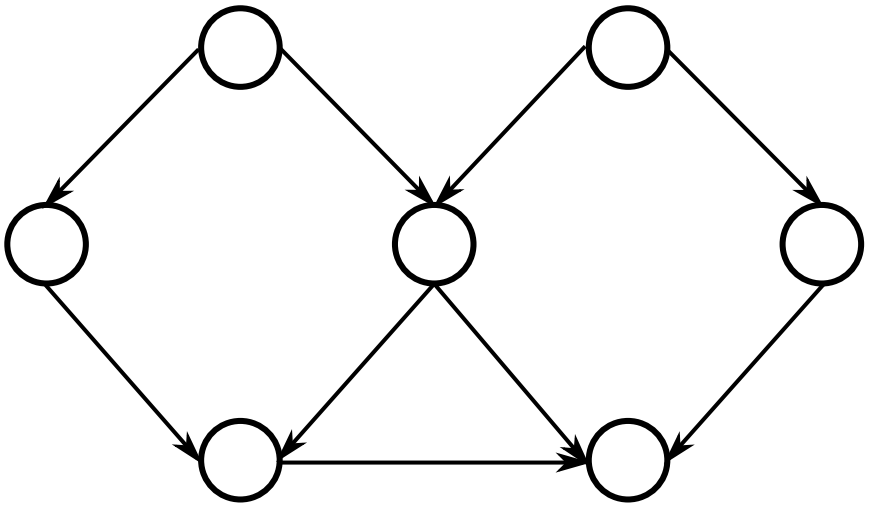


Directed acyclic graphs (DAGs)

- DAG: Directed graph which does not contain cycles.



directed graph with cycles



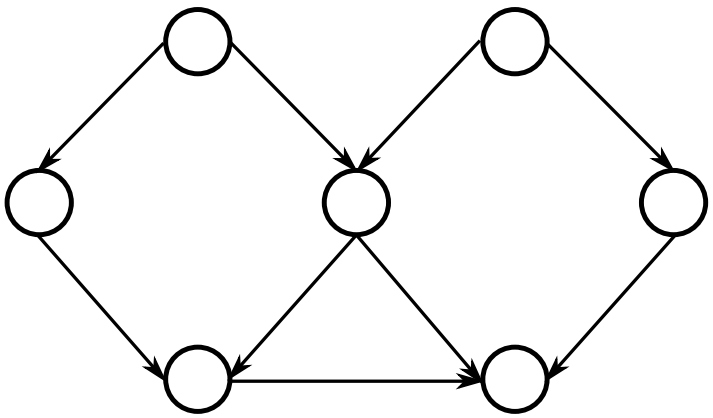
directed graph without cycles

DAG and distributions

- every DAG corresponds to a factorization of the joint probability into

$$P(X_1, \ldots, X_n) \quad = \quad \prod_{i=1}^{n} P(X_i \,|\, parents(X_i))$$

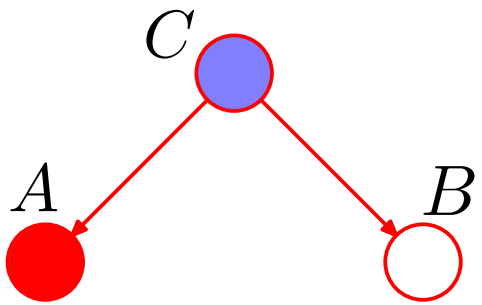
- efficient representation of the knowledge base (in terms of memory requirements)
  - topology: qualitative relationships
  - annotation: quantitative relationships



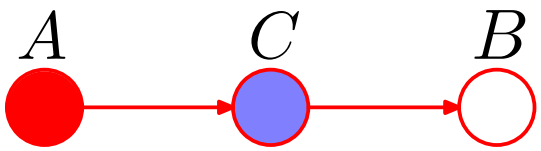
Conditional independence

- Node $A \perp X_i$ (red $\perp$ white) is **conditionally independent** of all nodes X_i given its Markov blanket (blue).
- **Markov blanket** of a node A: parents, children, and children's parents.
- Simple examples:

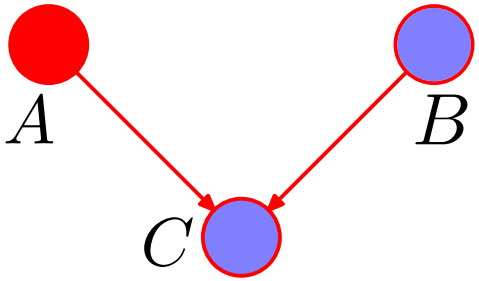
$C \in parents(A)$, $C \in children(A)$, $C \in children(A)$, $B \in parents(children(A))$



$A \perp B \mid C$

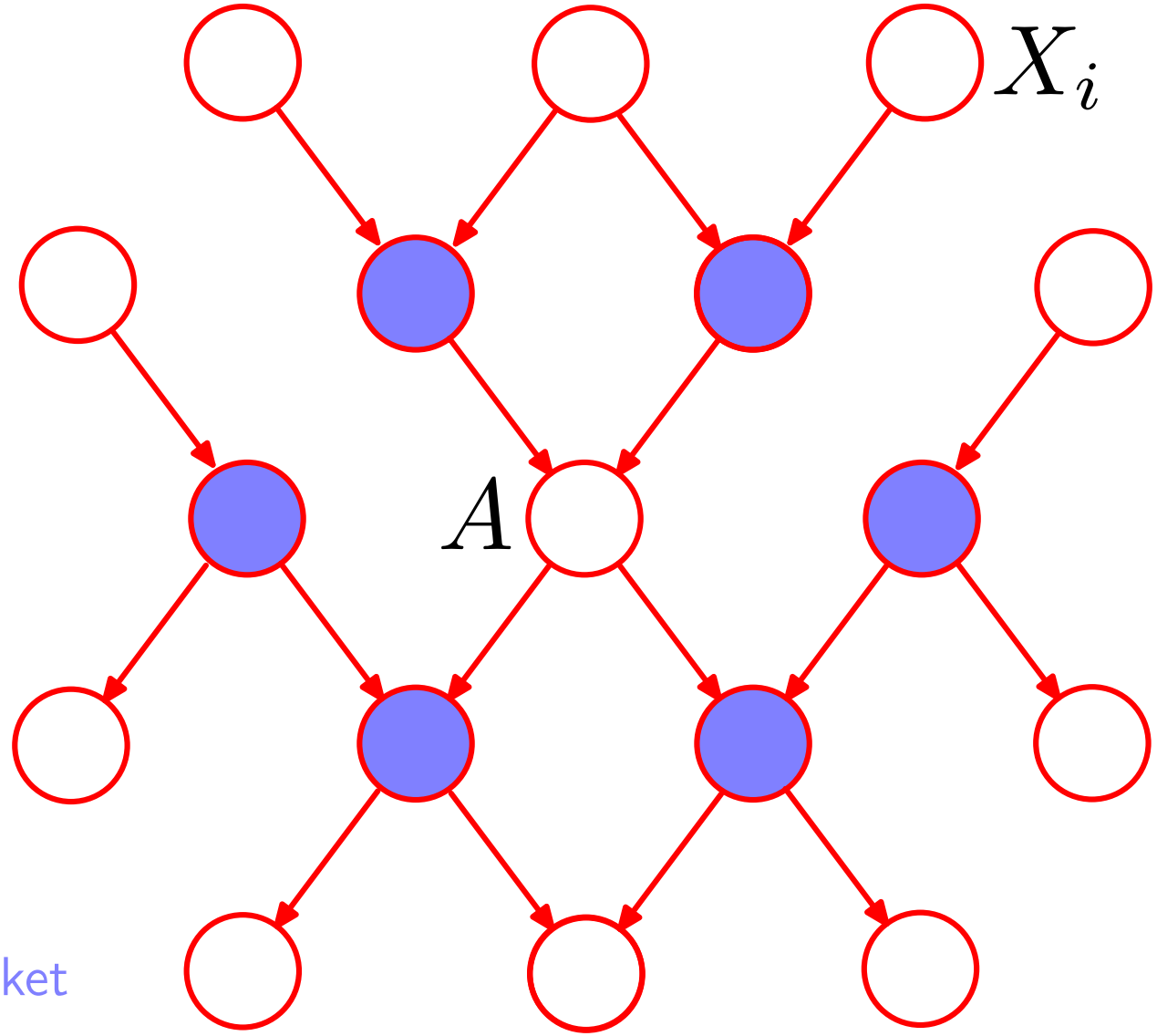


$A \perp B \mid C$



$A \not\perp B \mid C$

Markov blanket



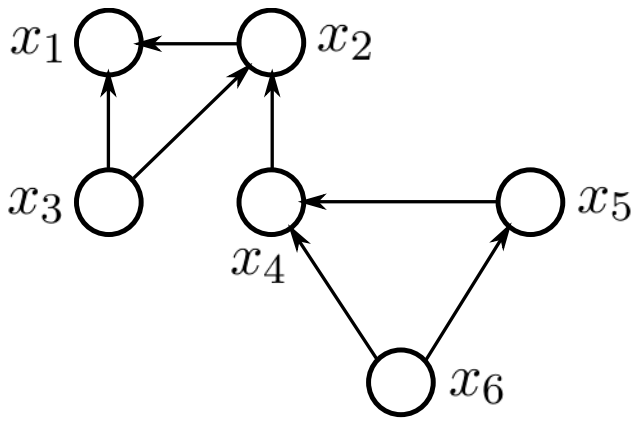
Markov blanket

Topological sorting

- Factorization of the unconditional probability:

$$P(X_1, \ldots, X_n) \quad = \quad \prod_{i=1}^{n} P(X_i \,|\, parents(X_i))$$

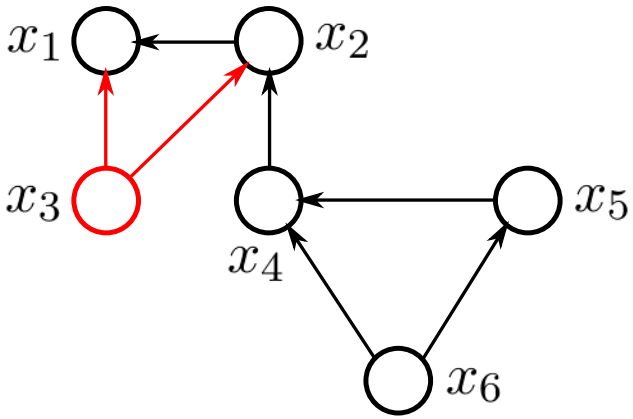
- Edges should always be directed from nodes with lower to nodes with higher indices (or vice versa). $\rightarrow$ Renumbering of nodes after reordering ("topological sorting").



- Topological sorting: Algorithmic concept:
  1. select and queue a node without parents
  2. delete that node from the DAG
  3. repeat until DAG is empty
  4. renumber nodes

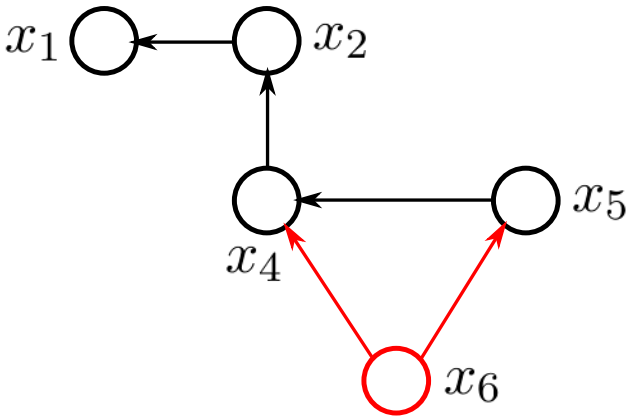
Topological sorting

- example:
  - x_3,



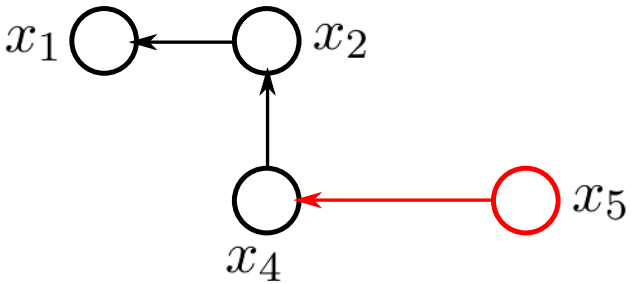
Topological sorting

- example:
  - $x_3, x_6,$



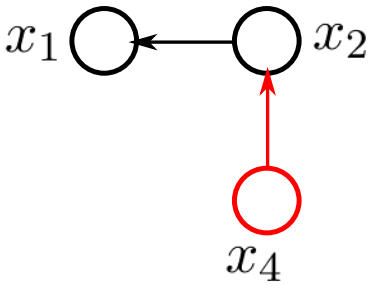
Topological sorting

- example:
  - $x_3, x_6, x_5,$



Topological sorting

- example:
  - $x_3, x_6, x_5, x_4,$

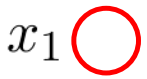


Topological sorting

- example:
  - $x_3, x_6, x_5, x_4, x_2,$

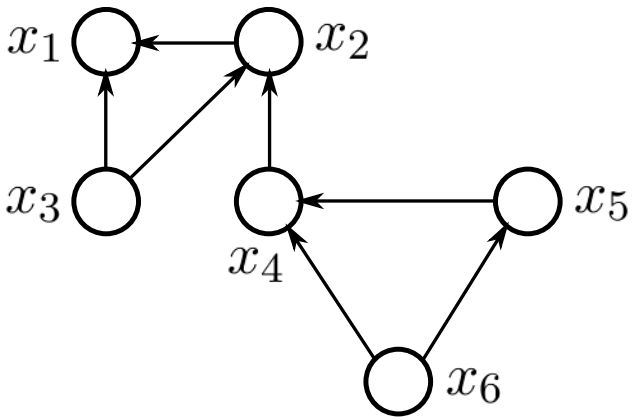
Topological sorting

- example:
  - $x_3, x_6, x_5, x_4, x_2, x_1$



Topological sorting

- example:
  - $x_3, x_6, x_5, x_4, x_2, x_1$
- other possible topological orderings:
  - $x_6, x_5, x_4, x_3, x_2, x_1$
  - $x_6, x_5, x_3, x_4, x_2, x_1$
  - $x_6, x_3, x_5, x_4, x_2, x_1$



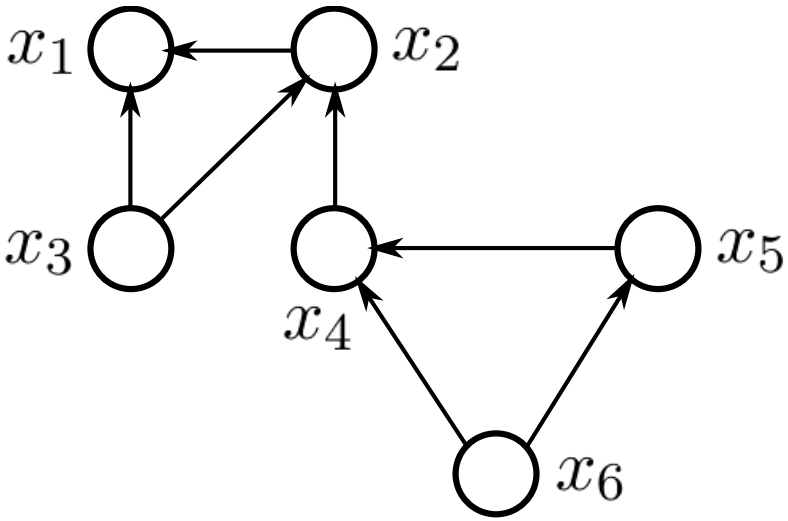
- choose a sorting and renumber
- Implementation: Kahn's algorithm or others (see supplementary material, part 6.5).

3.2.2 Inference on Bipartite Trees

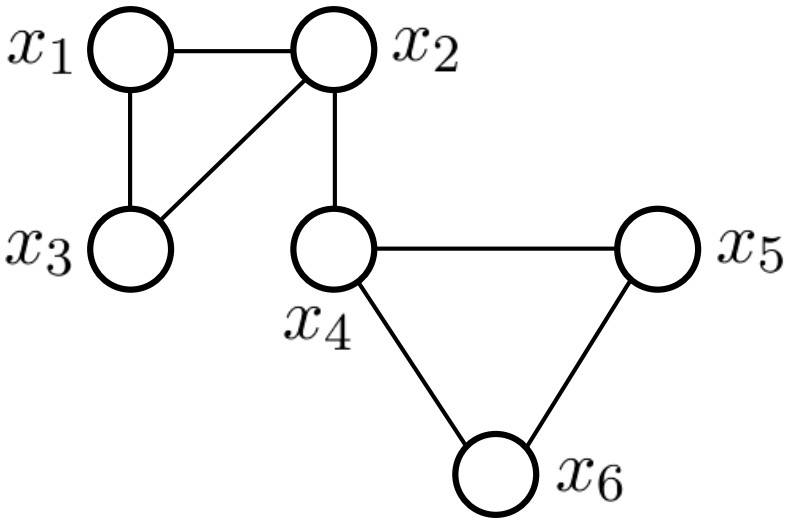
Undirected graphs

- **undirected graph:** directed graph with symmetric (undirected) edges

$$(x_i, x_j) \in K \quad \Rightarrow \quad (x_j, x_i) \in K$$



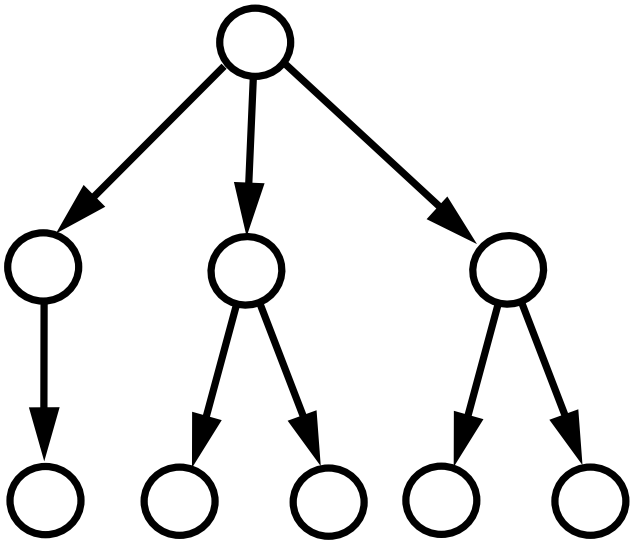
directed graph



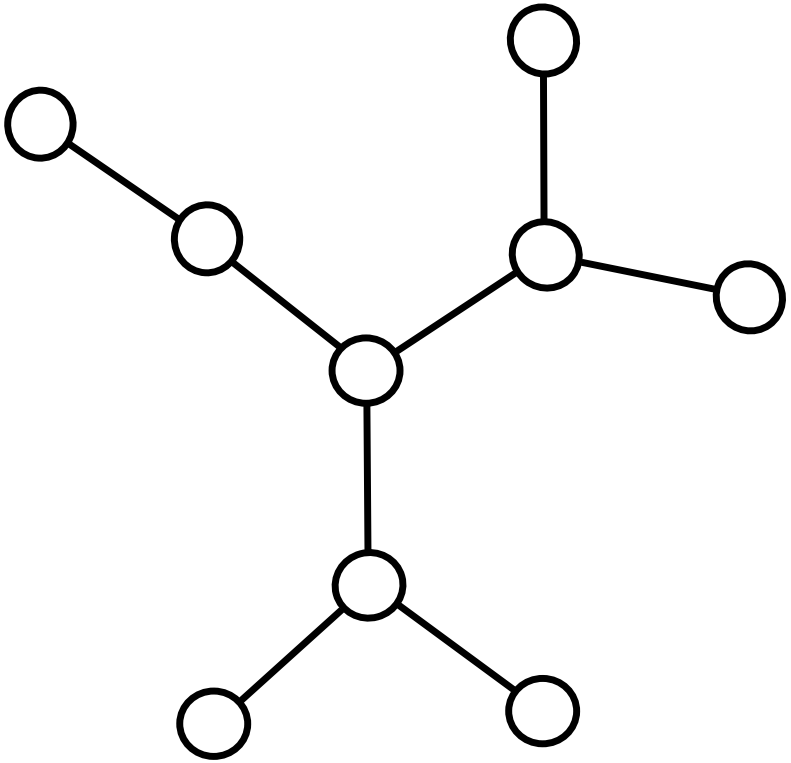
undirected graph

Trees

- **tree**: graph where each existing path between nodes is *unique*



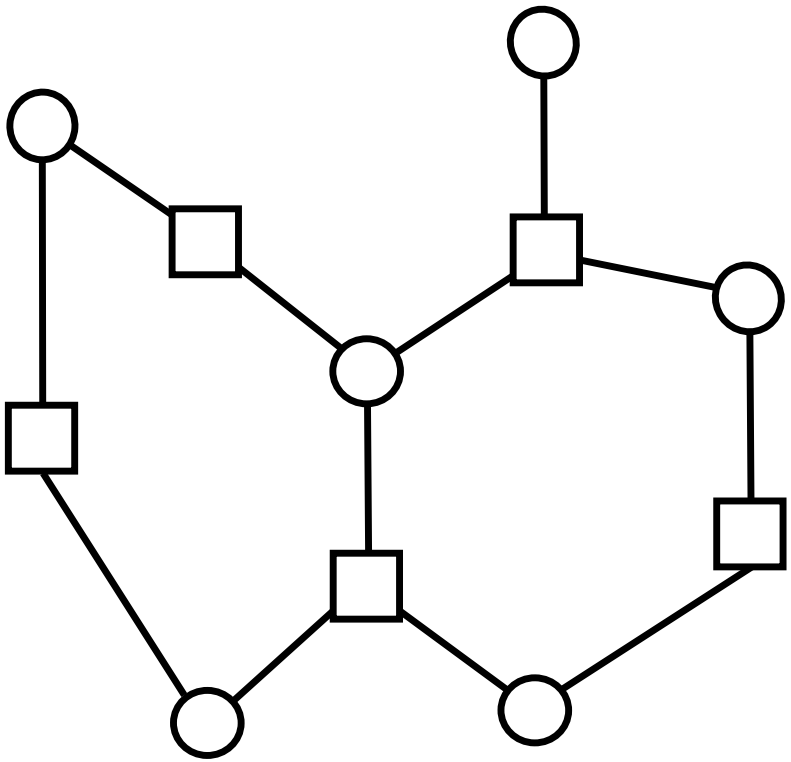
directed tree



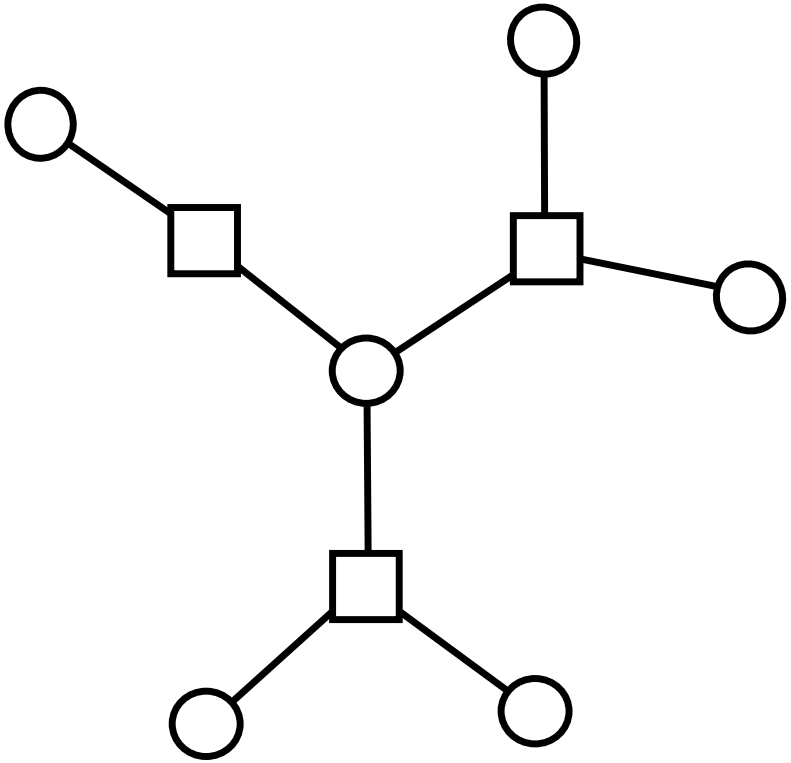
undirected tree

Bipartite graphs

- **bipartite** graph: two *types* of nodes (□ and ○)
  - each type can only connect to the other (□—○)

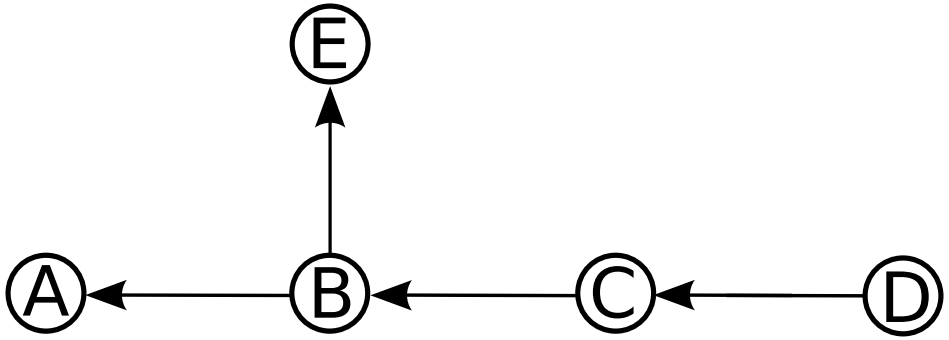


bipartite graph



bipartite tree

A simple (tree-shaped) DAG



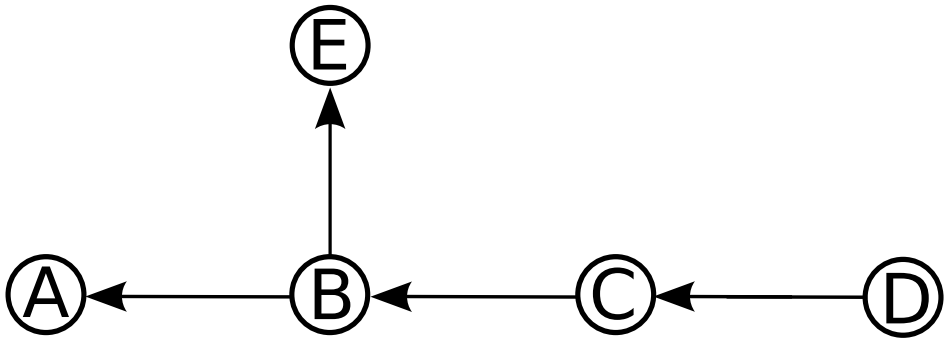
$$P(A,B,C,D,E) \quad = \quad P(A|B)\,P(E|B)\,P(B|C)\,P(C|D)\,P(D)$$

- **task**: update the degree of belief in A under the evidence $E{=}e^*$, i.e. compute $P(A|E{=}e^*)$

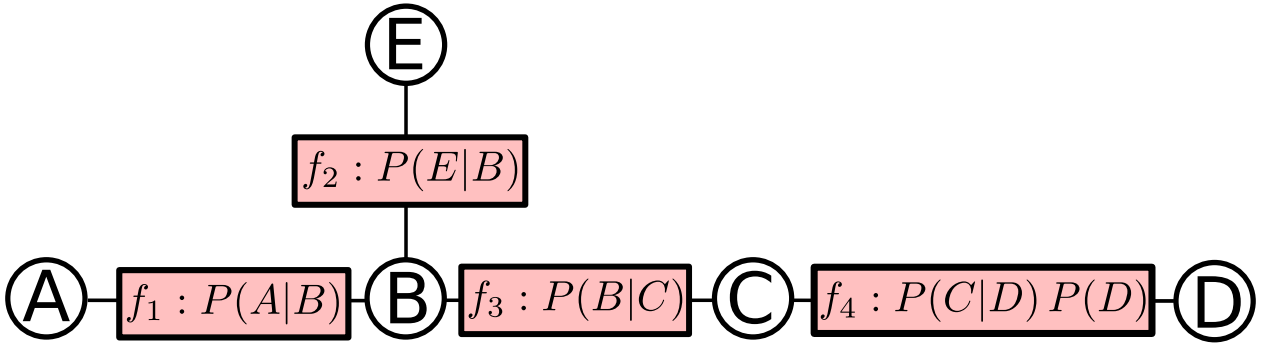
$$P(A|E{=}e^*) \;=\; \frac{P(A,E{=}e^*)}{P(E{=}e^*)} \;=\; \overbrace{\alpha\,P(A,E{=}e^*)}^{\text{normalization}} \;=\; \overbrace{\alpha \sum_{b,c,d} P(A,b,c,d,E{=}e^*)}^{\text{marginalization}}$$

$$= \;\alpha \sum_{b,c,d} P(A|b)\,P(E{=}e^*|b)\,P(b|c)\,P(c|d)\,P(d)$$

The bipartite tree

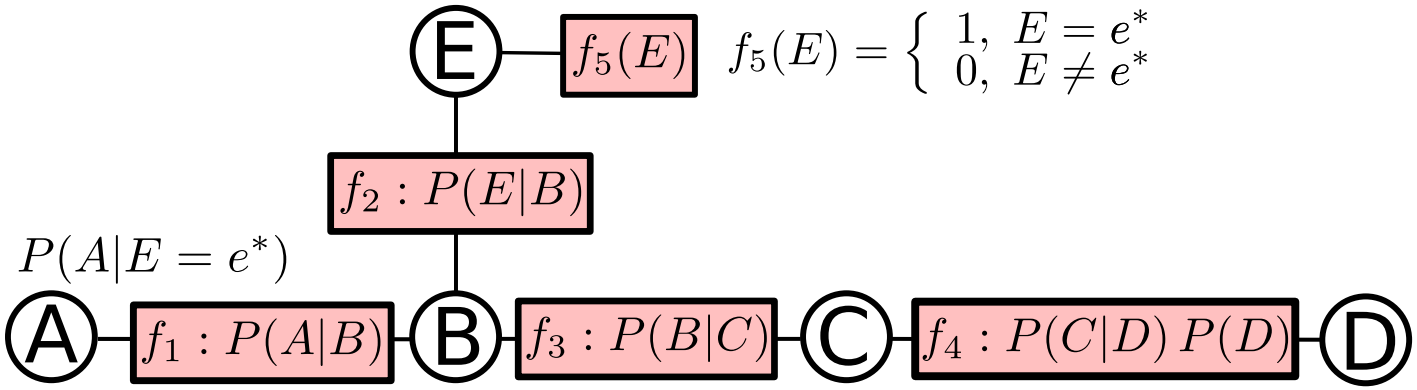


$$P(A,B,C,D,E) \quad = \quad P(A|B)P(E|B)P(B|C)P(C|D)P(D)$$



Inference

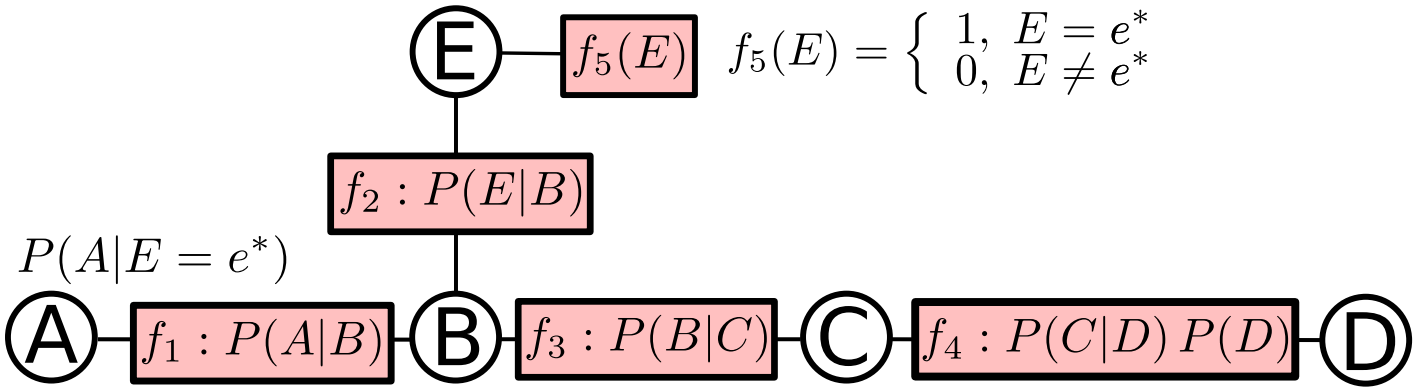
- Computation of $P(A|E{=}e^*)$



$$
\begin{aligned}
P(A|E{=}e^*) &= \alpha \sum_{b,c,d} P(A, b, c, d, E{=}e^*) \\
&= \alpha \sum_{b,c,d,e} P(A|b)\, P(e|b)\, P(b|c)\, P(c|d)\, P(d)\, f_5(e)
\end{aligned}
$$

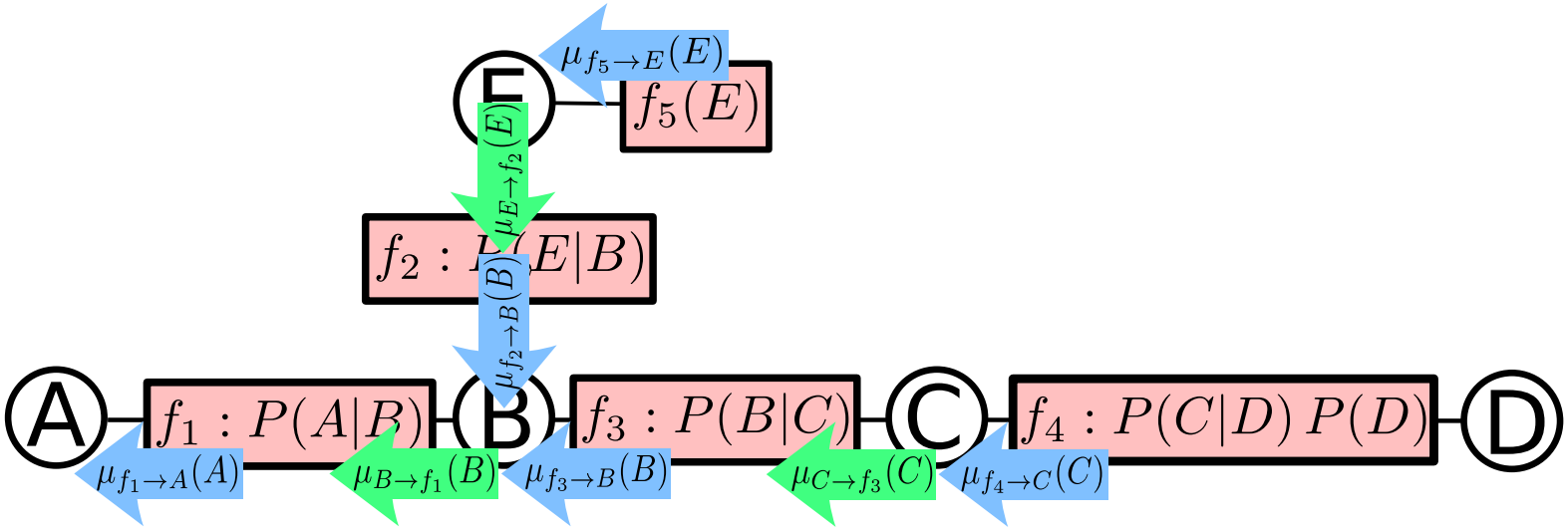
Inference

- Computation of $P(A|E{=}e^*)$



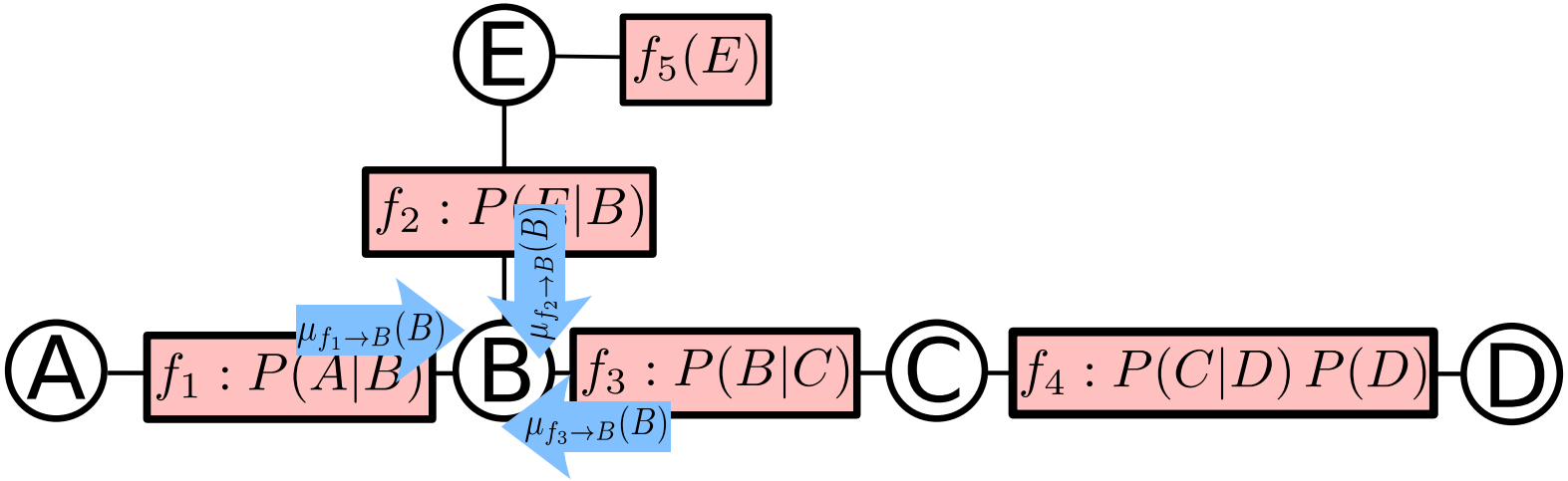
$$
\begin{aligned}
P(A|E{=}e^*) &= \alpha \sum_{b,c,d} P(A,b,c,d,E{=}e^*) \\
&= \alpha \sum_{b,c,d,e} P(A|b)\,P(e|b)\,P(b|c)\,P(c|d)\,P(d)\,f_5(e) \\
&= \alpha \sum_{b} P(A|b) \Big(\sum_{e} P(e|b)\,f_5(e) \Big) \Big(\sum_{c} P(b|c) \sum_{d} P(c|d)\,P(d) \Big)
\end{aligned}
$$

Message passing



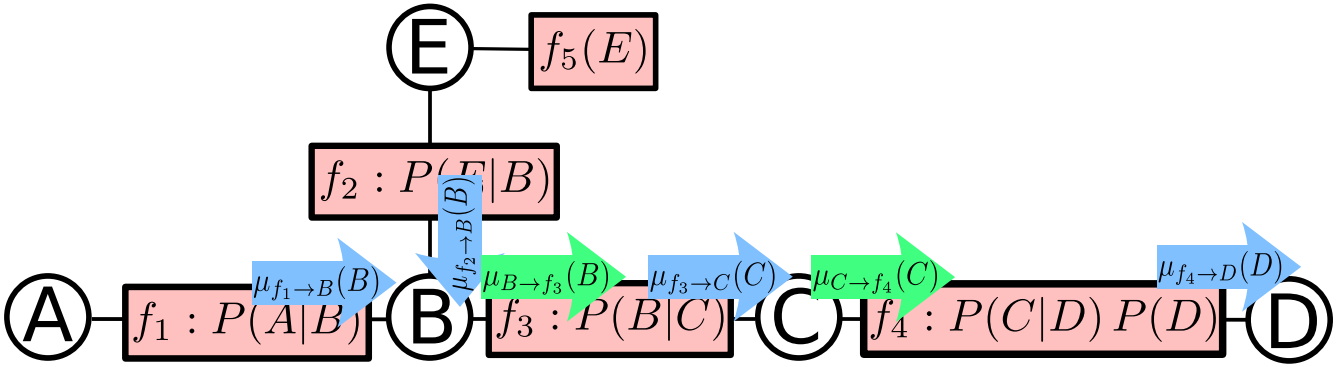
$$
\begin{aligned}
P(A|E{=}e^*) \quad = \quad \alpha \underbrace{\sum_b P(A|b) \overbrace{\underbrace{\Big(\sum_e P(e|b) \overbrace{\underbrace{f_5(e)}_{\mu_{f_5 \to E}(E)}}^{\mu_{E \to f_2}(E)}\Big)}_{\mu_{f_2 \to B}(B)} \underbrace{\Big(\sum_c P(b|c) \overbrace{\underbrace{\sum_d P(c|d)\,P(d)}_{\mu_{f_4 \to C}(C)}}^{\mu_{C \to f_3}(C)}\Big)}_{\mu_{f_3 \to B}(B)}}^{\mu_{B \to f_1}(B)}}_{\mu_{f_1 \to A}(A)}
\end{aligned}
$$

Message passing



$$
\begin{aligned}
P(B|E{=}e^*) &= \alpha \sum_{a,c,d,e} P(a|B)\,P(e|B)\,P(B|c)\,P(c|d)\,P(d)\,f_5(e) \\
&= \alpha \underbrace{\Big(\sum_a P(a|B)\Big)}_{\mu_{f_1 \to B}(B)} \underbrace{\Big(\sum_e P(e|B)\,f_5(e)\Big)}_{\mu_{f_2 \to B}(B)} \underbrace{\Big(\sum_c P(B|c) \sum_d P(c|d)\,P(d)\Big)}_{\mu_{f_3 \to B}(B)}
\end{aligned}
$$

Message passing

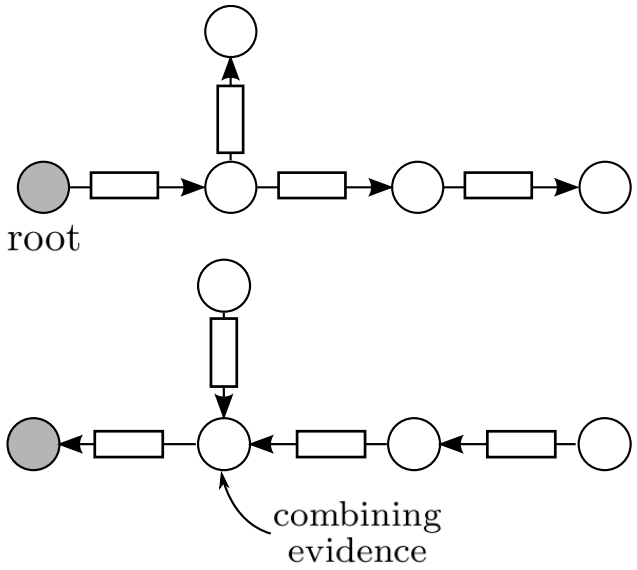


$$P(D|E{=}e^*) \quad = \quad \alpha \sum_{a,b,c,e} P(a|b)P(e|b)P(b|c)P(c|D)P(D)f_5(e)$$

$$= \quad \alpha \underbrace{\sum_c P(c|D)\,P(D) \overbrace{\underbrace{\sum_b P(b|c) \overbrace{\underbrace{\sum_e P(e|b)\,f_5(e)}_{\mu_{f_2\to B}(B)} \underbrace{\sum_a P(a|b)}_{\mu_{f_1\to B}(B)}}^{\mu_{B\to f_3}(B)}}_{\mu_{f_3\to C}(C)}}^{\mu_{C\to f_4}(C)}}_{\mu_{f_4\to D}(D)}$$

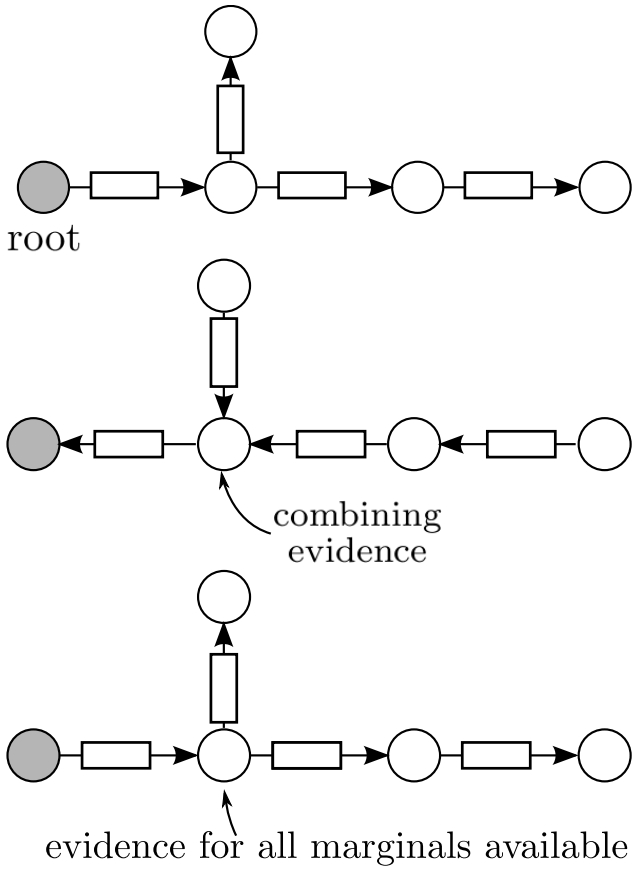
Message passing

- first pass from root to leaves: "request"
- second pass from leaves to root: "collect"

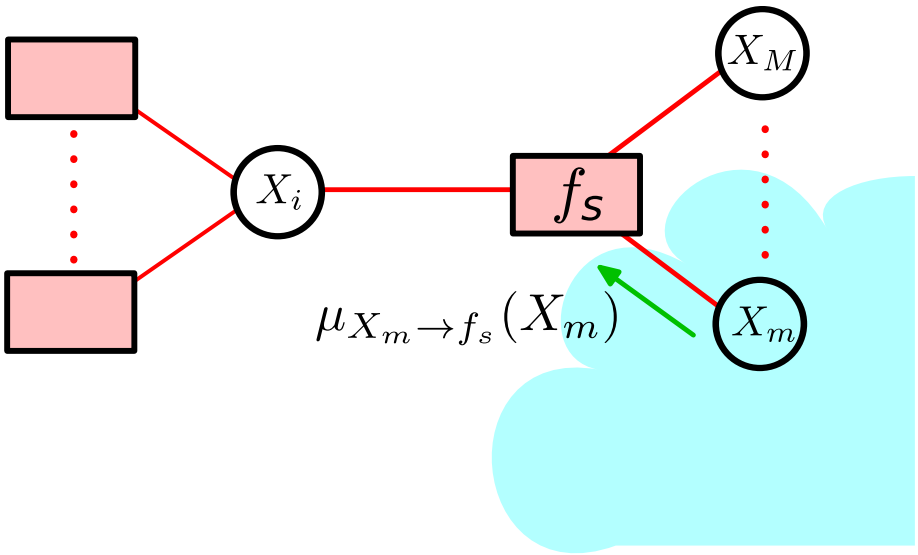


Message passing

- first pass from root to leaves: "request"
- second pass from leaves to root: "collect"
- a third pass can calculate all other marginals: "distribute"



The sum-product algorithm

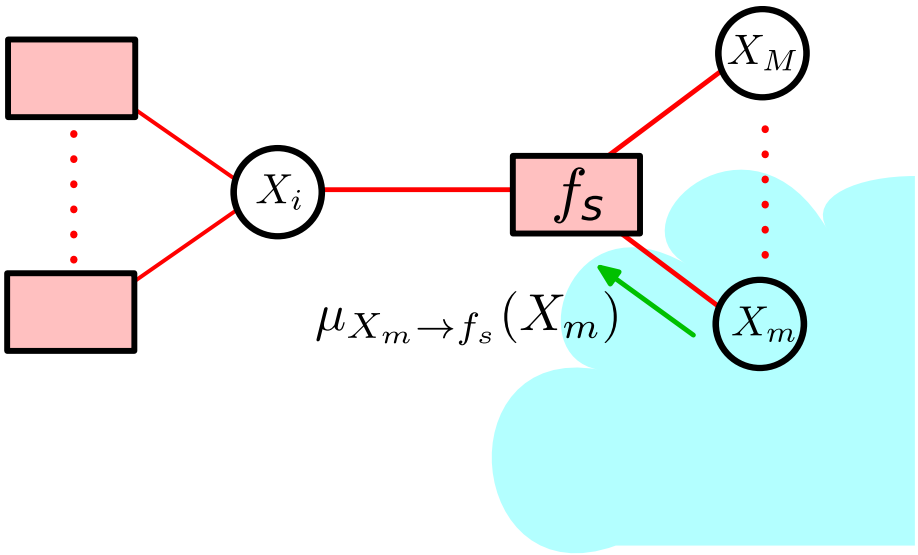


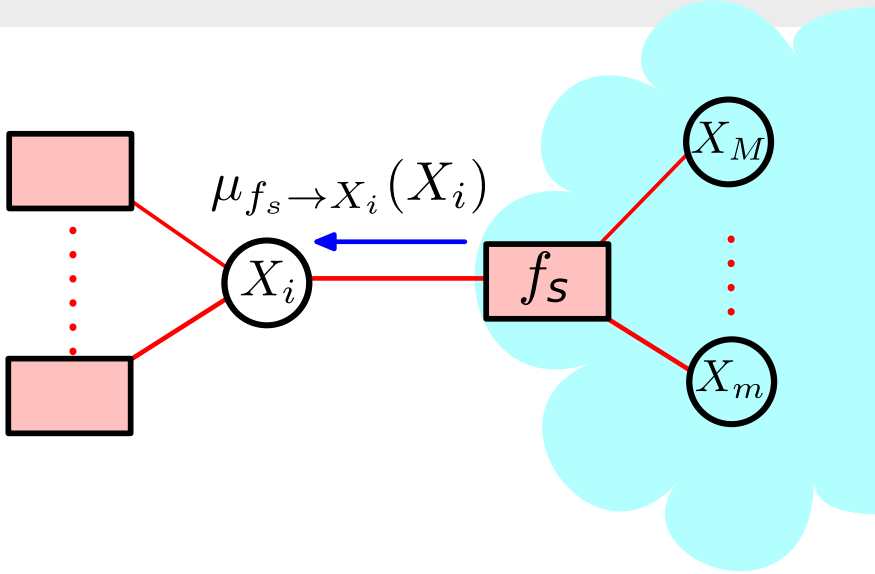
- product message from X_m to f_s

$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m) \qquad \text{(product)}$$

Bishop, 2006, p. 402

The sum-product algorithm





- product message from X_m to f_s
- sum message from f_s to X_i

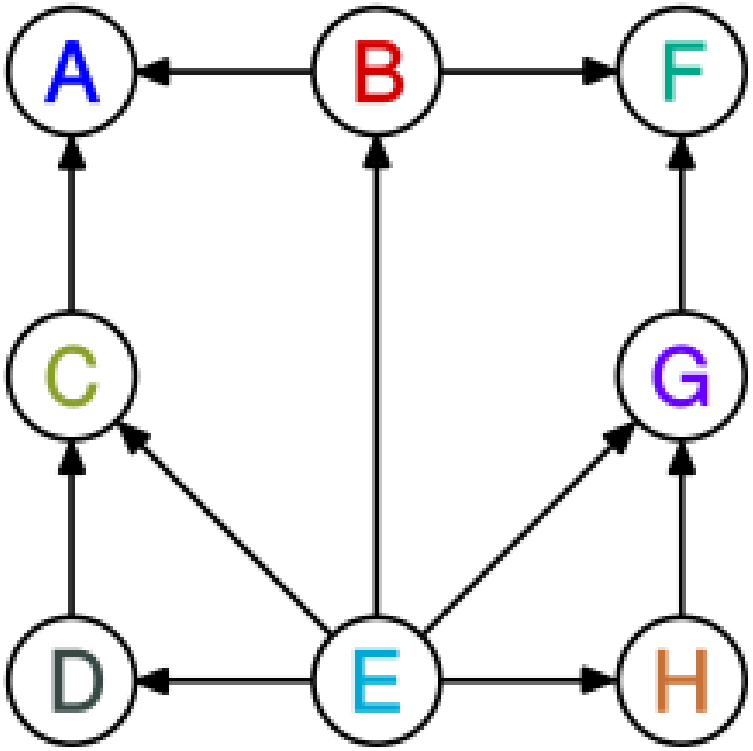
$$\mu_{X_m \to f_s}(X_m) := \prod_{l \in \text{neighbor}(X_m) \setminus \{f_s\}} \mu_{f_l \to X_m}(X_m) \qquad \text{(product)}$$

$$\mu_{f_s \to X_i}(X_i) := \sum_{x_m, \ldots, x_M} f_s(X_i, x_m, \ldots, x_M) \prod_{k \in \text{neighbor}(f_s) \setminus \{X_i\}} \mu_{X_k \to f_s}(x_k) \qquad \text{(sum)}$$

Bishop, 2006, p. 402

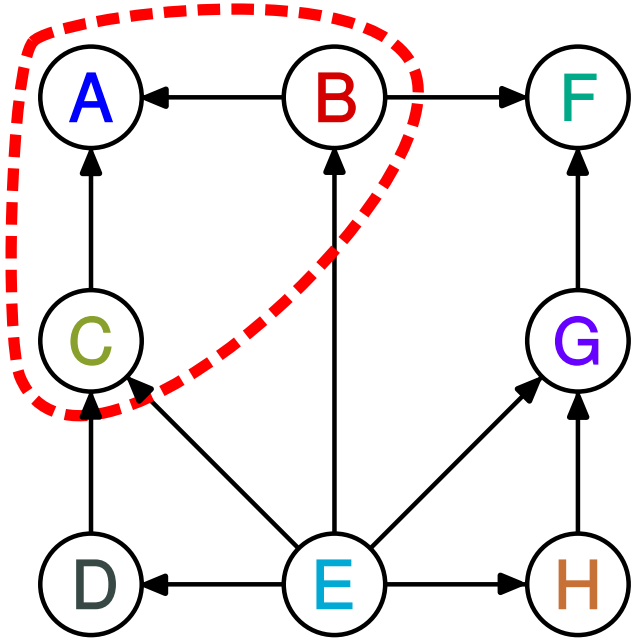
Junction trees

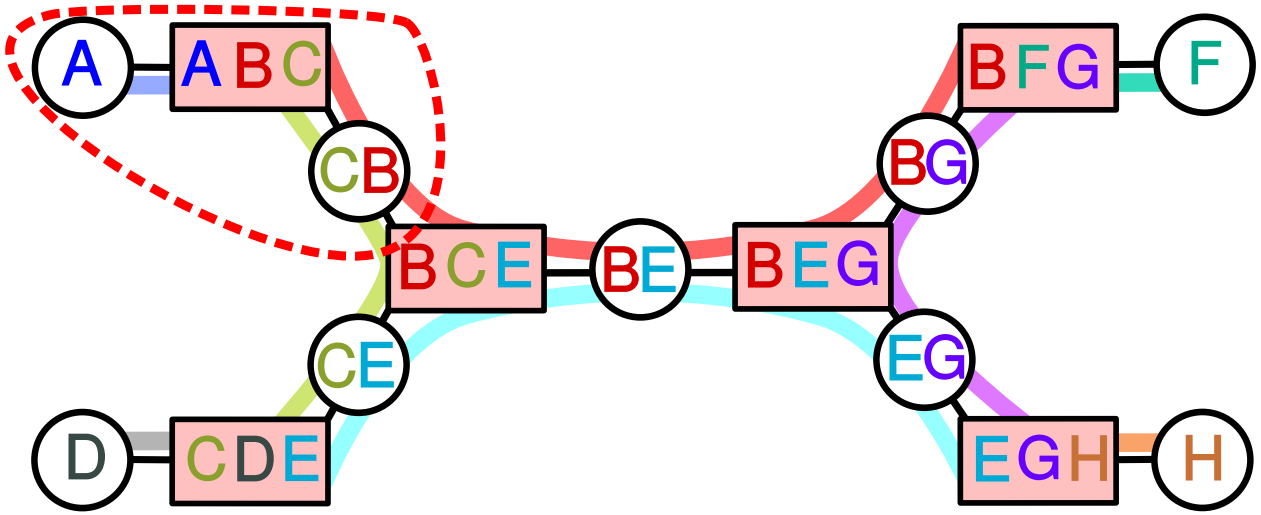
- efficient inference requires trees
- DAGs may not be trees



Junction trees

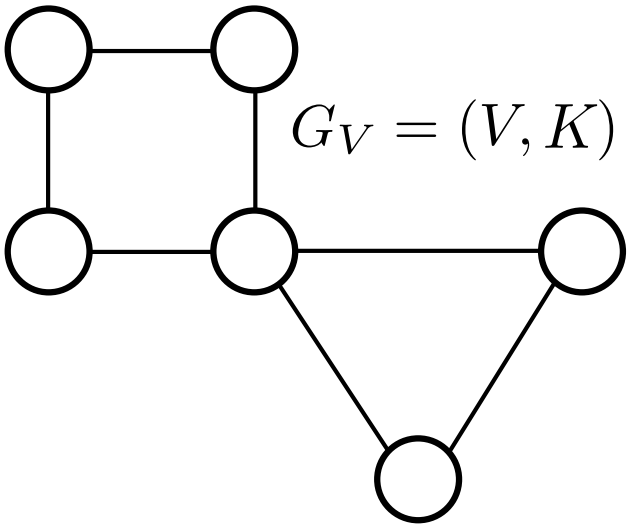
- efficient inference requires trees
- DAGs may not be trees
- hide sets of nodes which violate the tree property within **cliques**
- construct a tree based on cliques





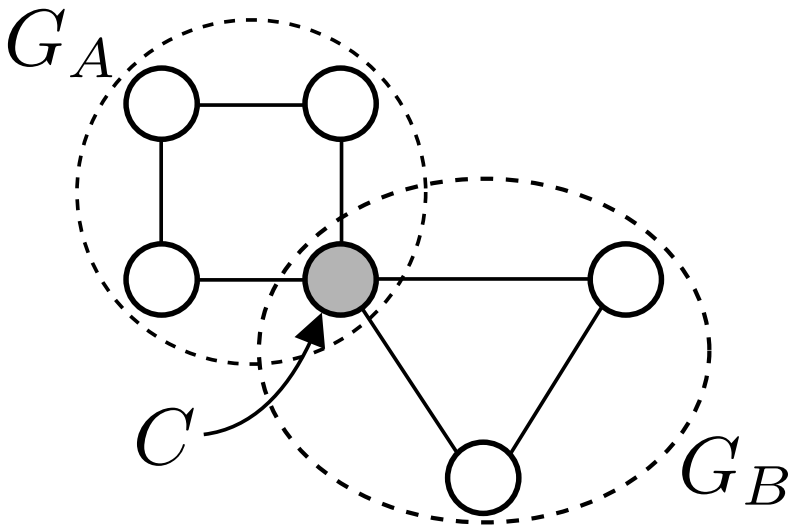
3.2.3 Decomposable Undirected Graphs

Undirected Graph



undirected graph G_V

Separators



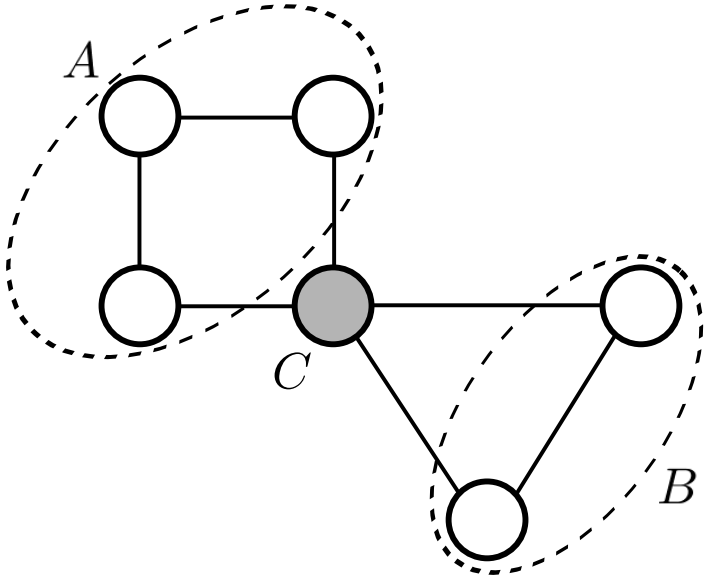
node C separates subsets A and B

Separator

A set C of nodes **separates** two undirected subgraphs G_A and G_B if *every* path from node set A to node set B has to pass through an element of C.

Decomposable graphs

- A, B, C are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - A, B, C are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - C separates G_A and G_B, and
  - subgraph G_C is complete.

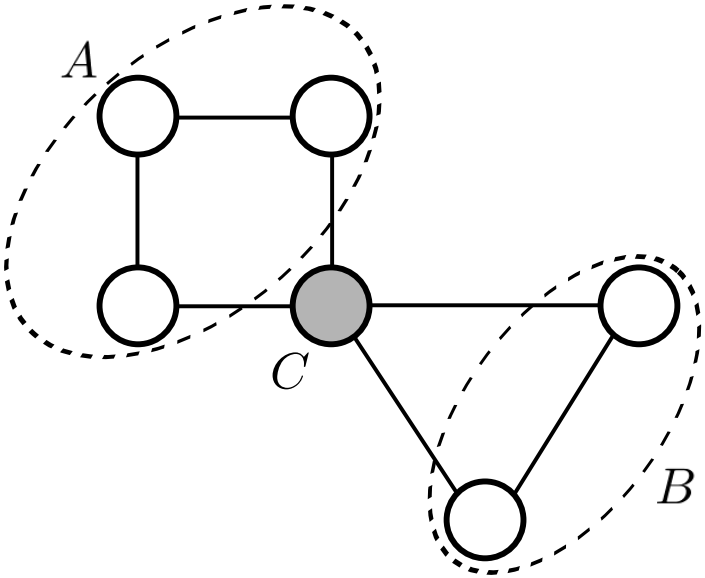


a proper decomposition

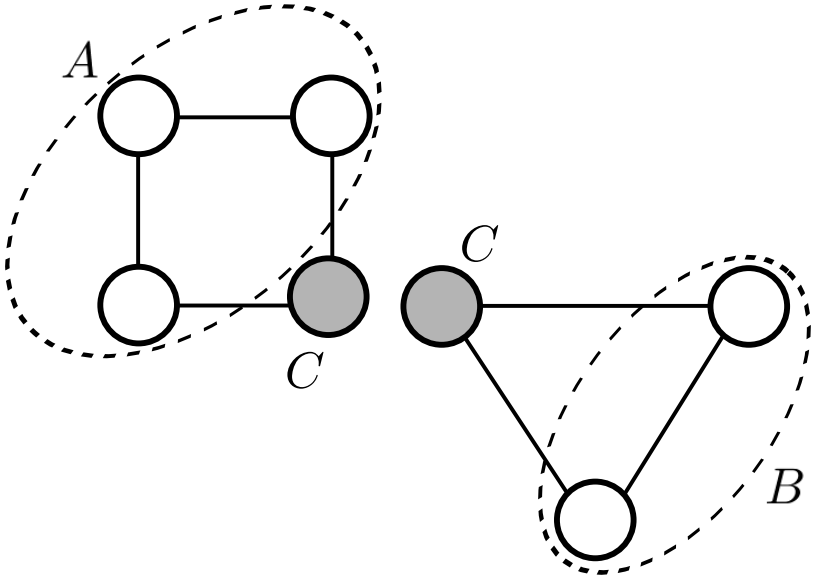
Decomposable graphs

- A, B, C are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - A, B, C are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - C separates G_A and G_B, and
  - subgraph G_C is complete.

- G is **decomposable** if it is complete, or a proper decomposition A, B, C exist, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.



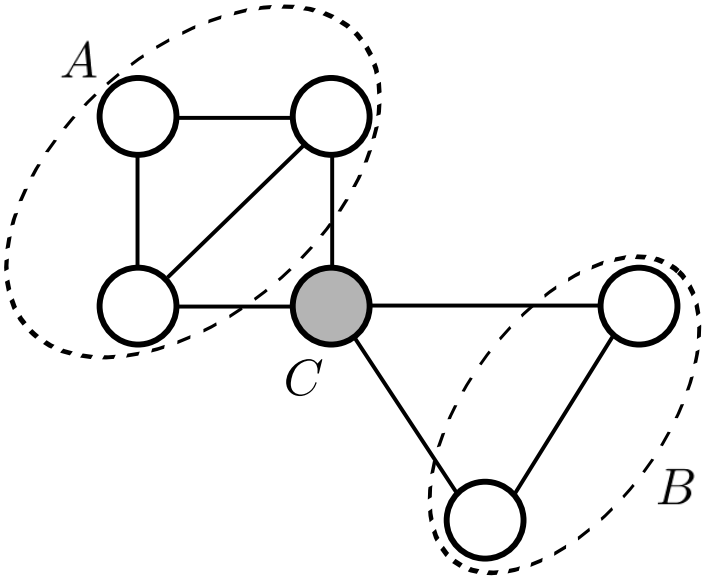
not a decomposable graph



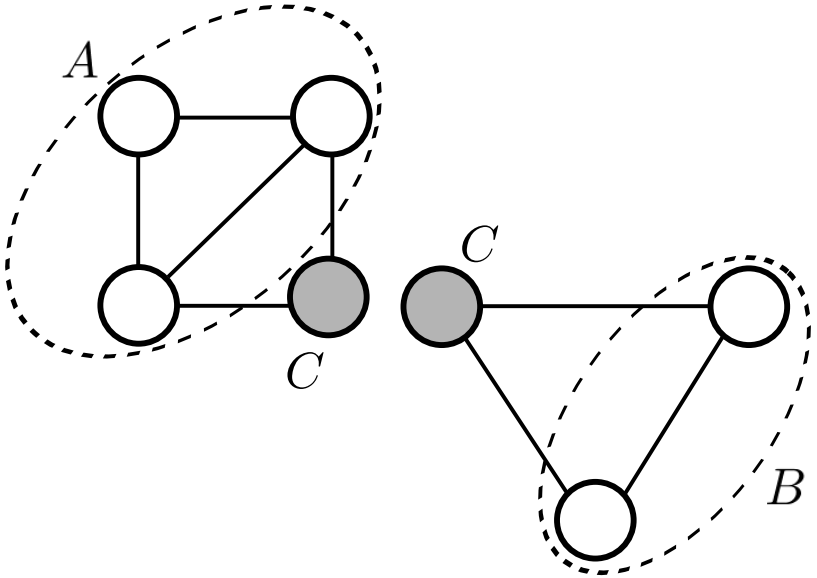
$G_{A \cup C}$ is not decomposable

Decomposable graphs

- A, B, C are a **proper decomposition** of an undirected graph $G = (V, K)$ if:
  - A, B, C are non-empty and disjoint subsets with $V = A \cup B \cup C$,
  - C separates G_A and G_B, and
  - subgraph G_C is complete.
- G is **decomposable** if it is complete, or a proper decomposition A, B, C exist, where $G_{A \cup C}$ and $G_{B \cup C}$ are decomposable.



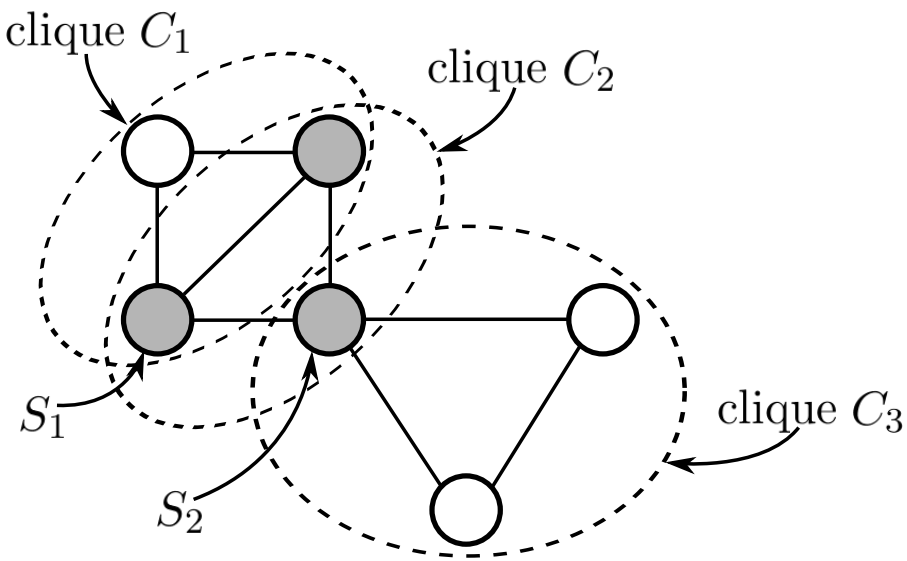
a decomposable graph



$G_{A \cup C}$ is decomposable and $G_{B \cup C}$ is complete

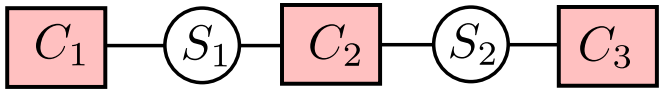
Cliques and separators

- **cliques** are maximally complete subgraphs
- decomposable graphs can be decomposed into cliques and separators
- cliques and separators form a bipartite graph



a decomposable graph

$\rightsquigarrow$



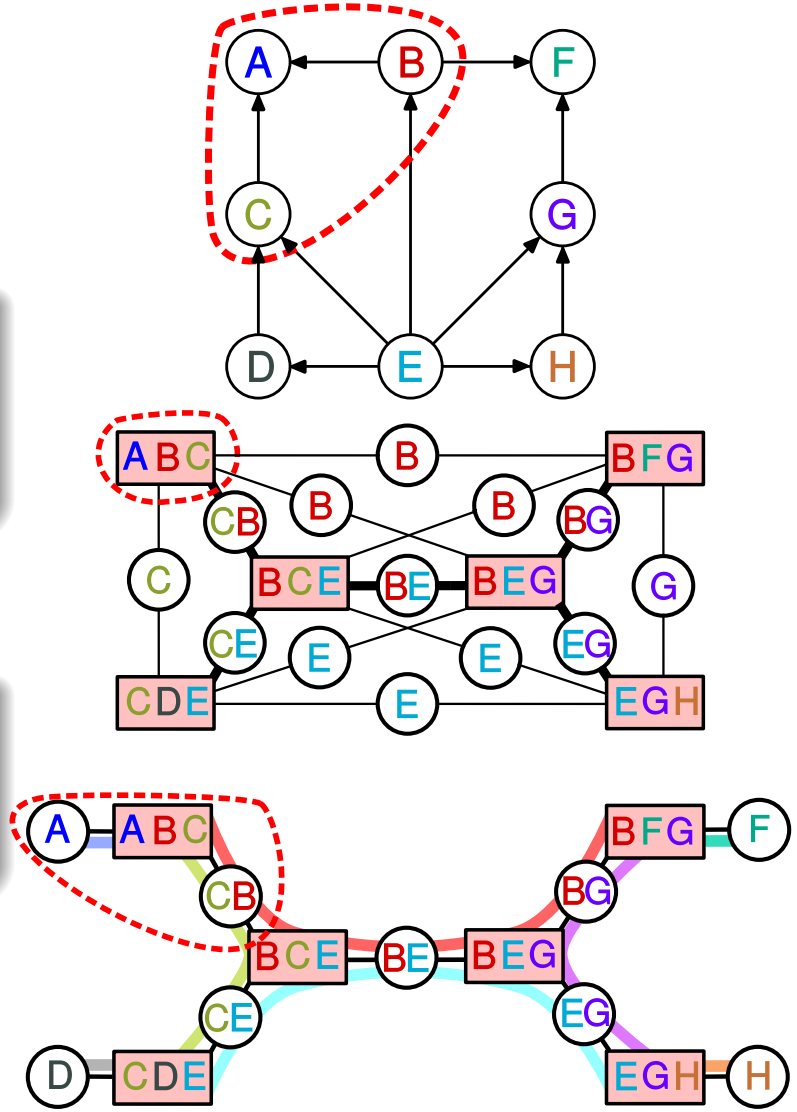
Junction trees

Existence Cowell et al. (1999)

There exist a junction tree of cliques for the graph $\mathcal{G}$ if and only if $\mathcal{G}$ is **decomposable**.

Running intersection property

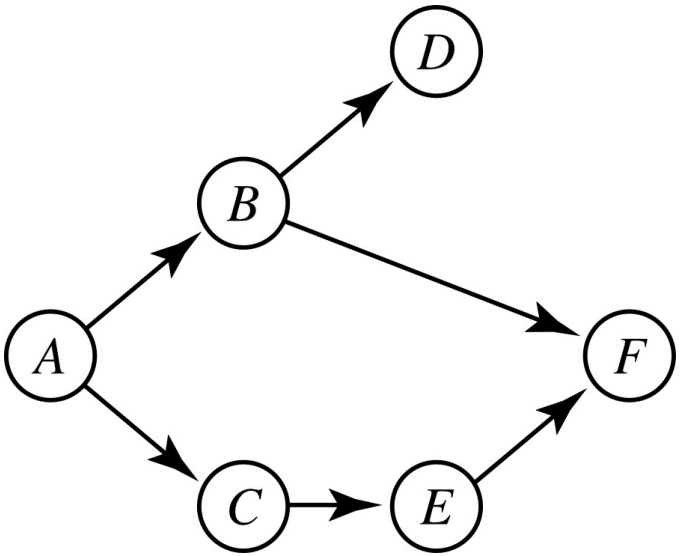
All nodes on the path between two cliques, which both contain variable X, also contain X.



3.2.4 Construction of the Junction Tree

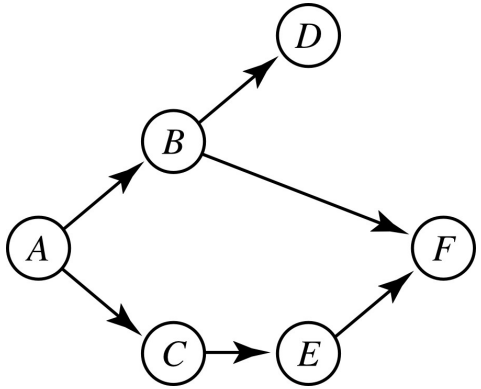
Define the knowledge base

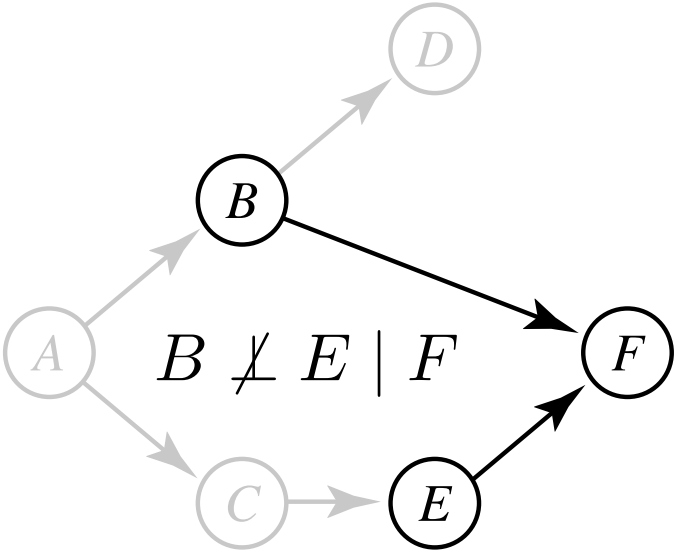
1. construct the DAG
   - perform topological sorting, e.g.: A B C E D F

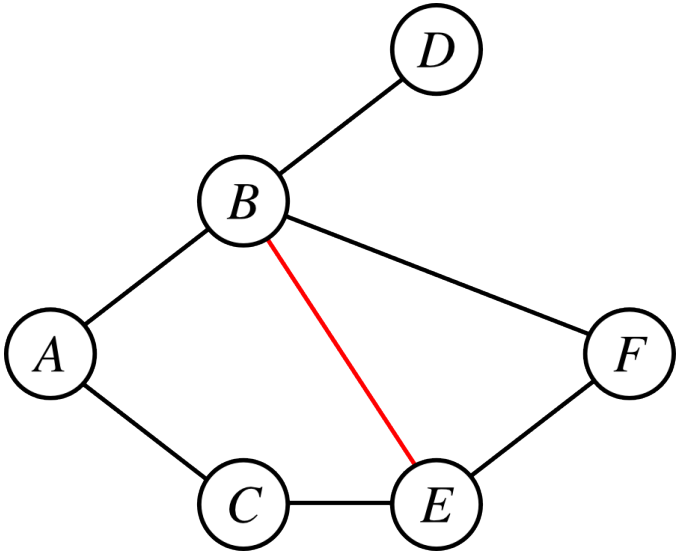


Construction of the undirected decomposable graph

1. construct the DAG
2. convert to the (undirected) moral graph
   - insert undirected edges between parents of nodes which are not yet connected
   - convert all directed to undirected edges
   - conditional independence assertions can now be read off the undirected graph

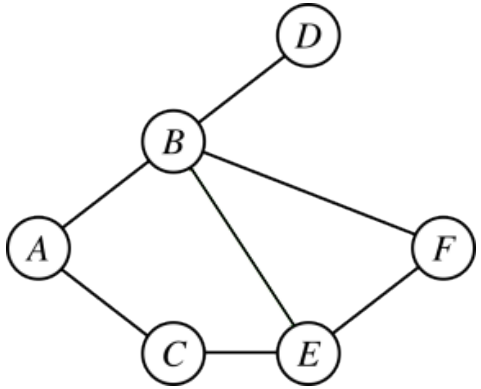


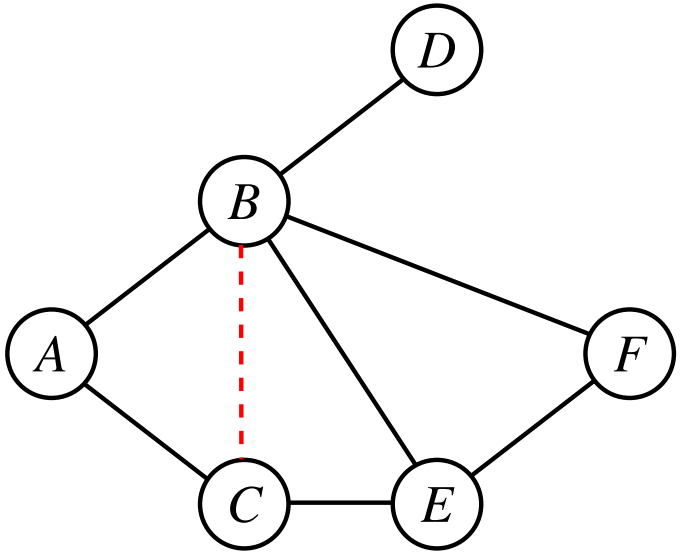


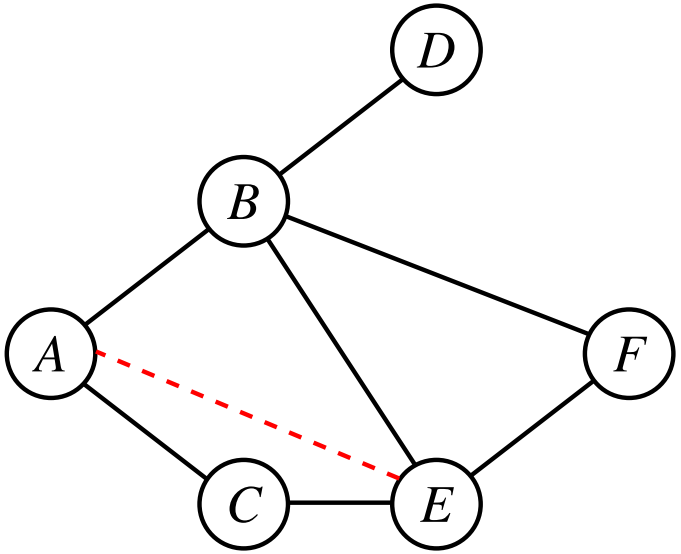


Construction of the undirected decomposable graph

1. construct the DAG
2. convert to the (undirected) moral graph
3. construct a chordal (decomposable) graph
   - add chords to all circles of length 4+ ("shortcuts")
   - chordal (decomposable) graphs not unique

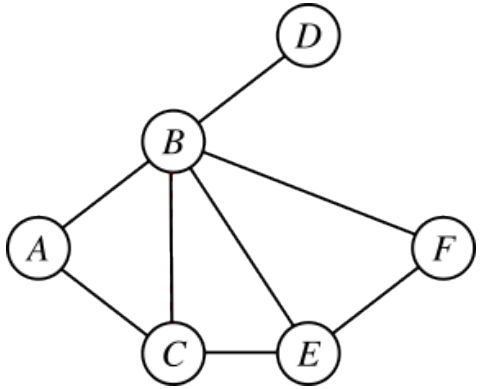


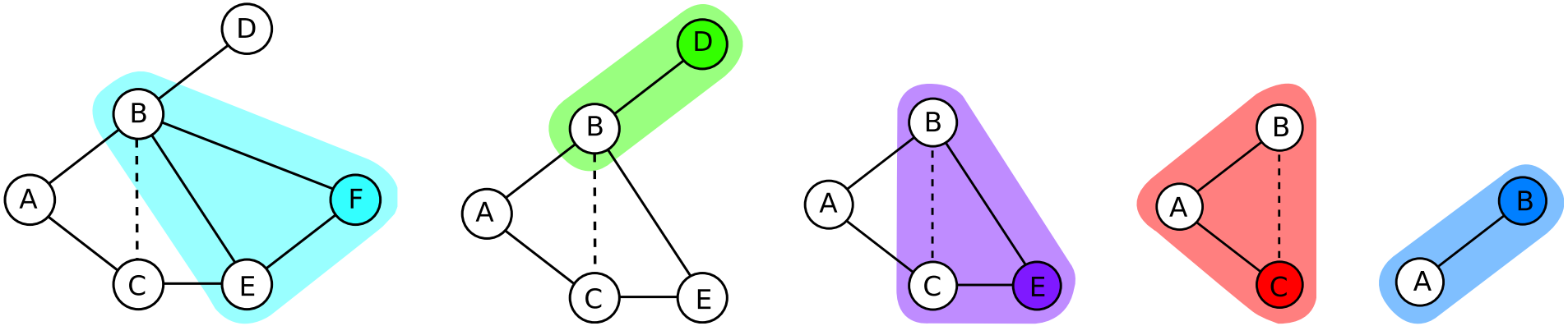




Identification of cliques and separators

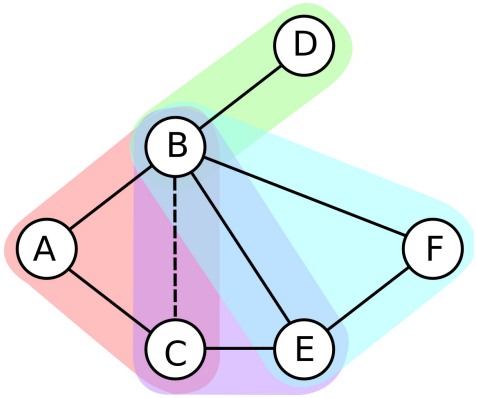
1. construct the DAG
2. convert to the (undirected) moral graph
3. construct a chordal (decomposable) graph
4. identify cliques
   - cliques are maximally complete subgraphs
   - cliques can be found by elimination
   - requires variable ordering by topological sorting: F, D, E, C, B, A
   - The elimination step may lead to complete subgraphs which are nested (cf. dark blue and red subgraphs below). Subgraphs which are not maximally complete must be omitted.

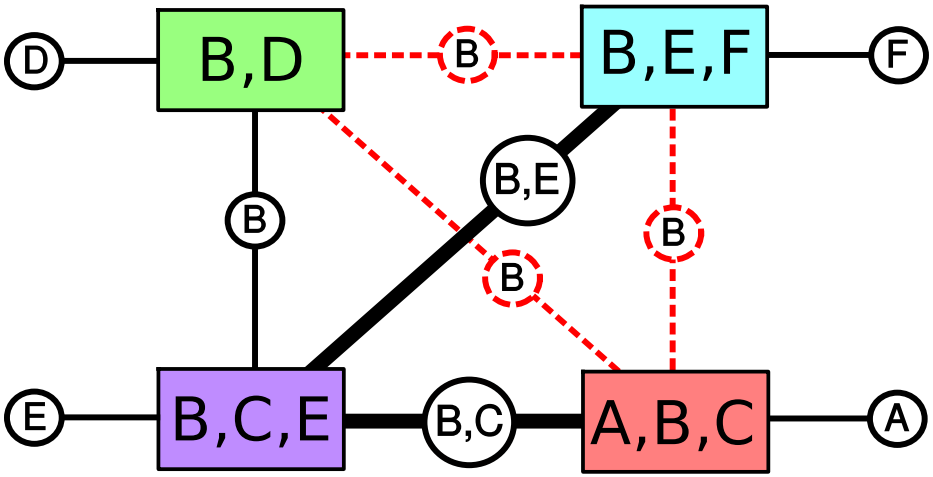




Construction of the junction tree

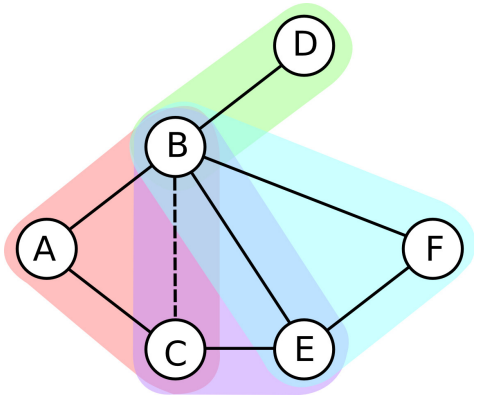
1. construct the DAG
2. convert to the (undirected) moral graph
3. construct a chordal (decomposable) graph
4. identify cliques
5. construct bipartite graph
   - edges weighted by separator size → number of nodes within separator
   - find a *maximal spanning tree*

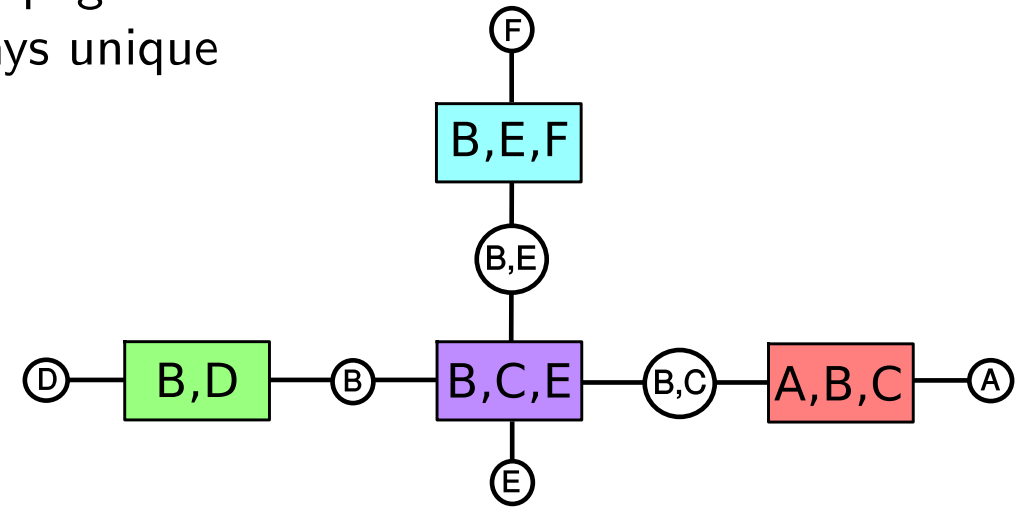




Construction of the junction tree

1. construct the DAG
2. convert to the (undirected) moral graph
3. construct a chordal (decomposable) graph
4. identify cliques
5. construct bipartite graph
6. junction tree for inference
   - maximal spanning tree of the clique graph
   - data structure for belief propagation
   - not always unique



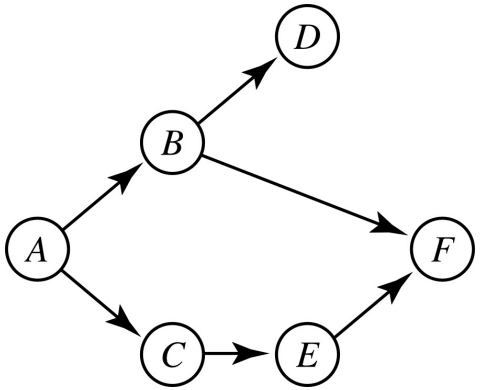


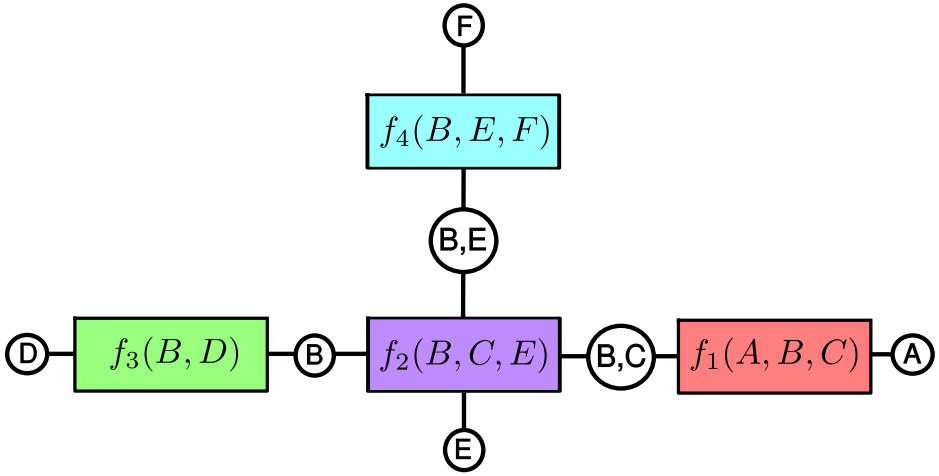
Inference

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$

$f_1(A,B,C)$	$f_2(B,C,E)$	$f_3(B,D)$	$f_4(B,E,F)$
$P(C\|A)\,P(B\|A)\,P(A)$	$P(E\|C)$	$P(D\|B)$	$P(F\|B,E)$

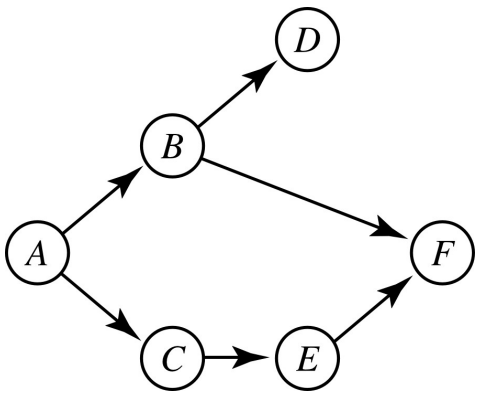
$$P(A,B,C,D,E,F) \;=\; P(A)\,P(B|A)\,P(C|A)\,P(D|B)\,P(E|C)\,P(F|B,E)$$

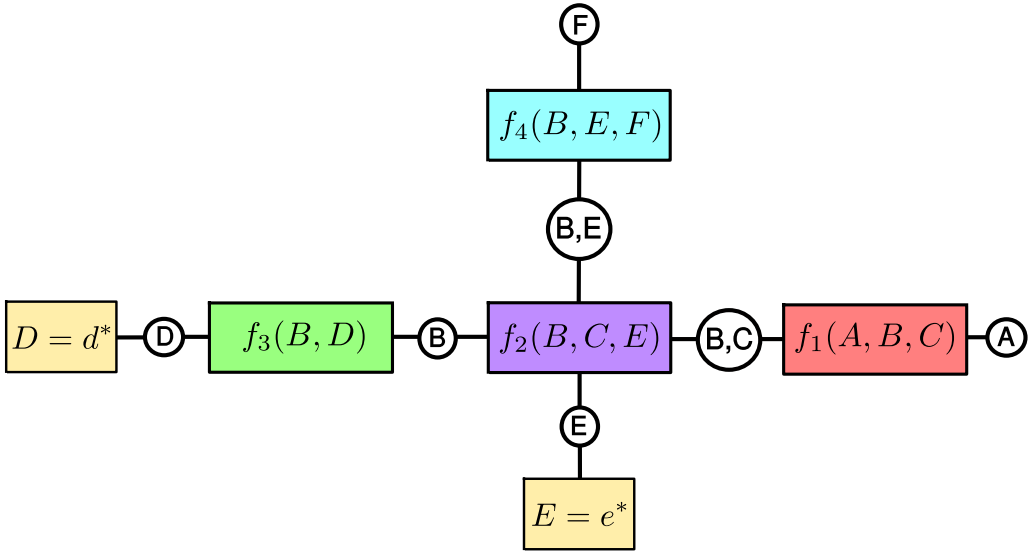




Inference

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
2. modification of the clique potentials by the observed evidence
   - for each observation $Y = y^*$ find *one* f_k with $Y \in \mathcal{X}_k$
   - add a node $f_k(Y) = \begin{cases} 1, & Y = y^* \\ 0, & Y \neq y^* \end{cases}$
   - example: $D = d^*$ and $E{=}e^*$

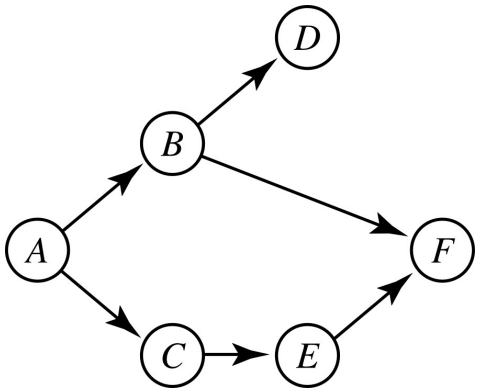


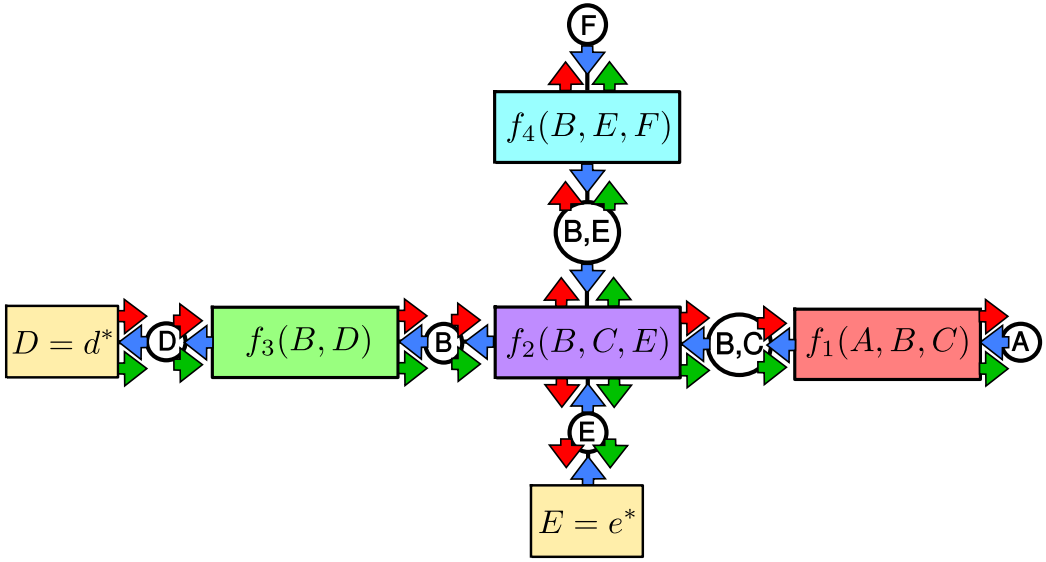


Inference

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
2. modification of the clique potentials by the observed evidence
3. message passing
   - begin "request" pass from arbitrary node N, e.g. $D = d^*$
   - wait for all message of "collect" pass to return
   - send last "distribute" pass from N

   $\Rightarrow$ all marginals are computed simultaneously

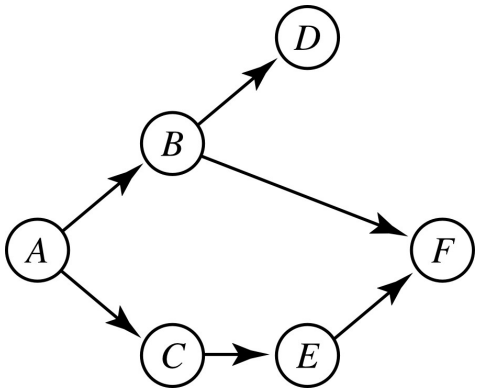


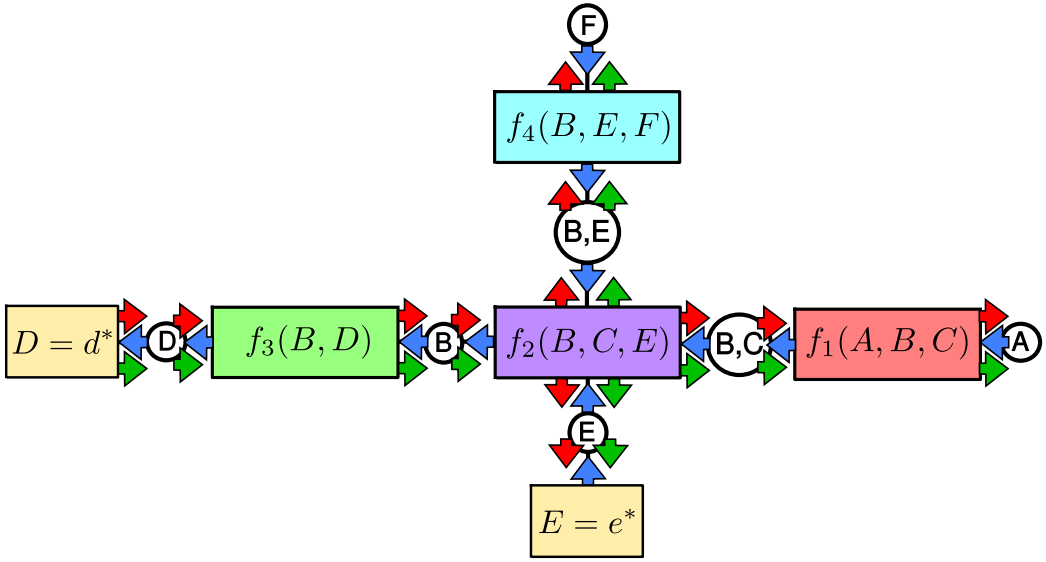


Inference

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
2. modification of the clique potentials by the observed evidence
3. message passing
4. calculate marginals from messages

$$P(C,\, D=d^*, E{=}e^*) = \sum_b P(C, b, D=d^*, E{=}e^*) = \sum_b \mu_{f_1\to BC}(b,C)\cdot\mu_{f_2\to BC}(b,C)$$

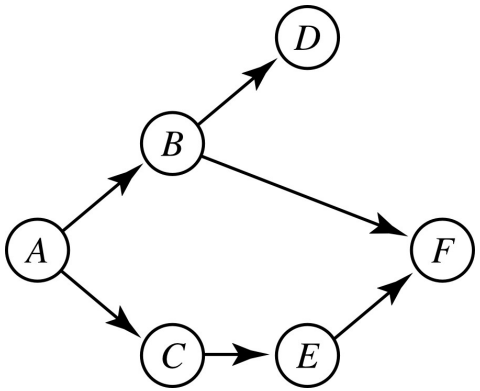


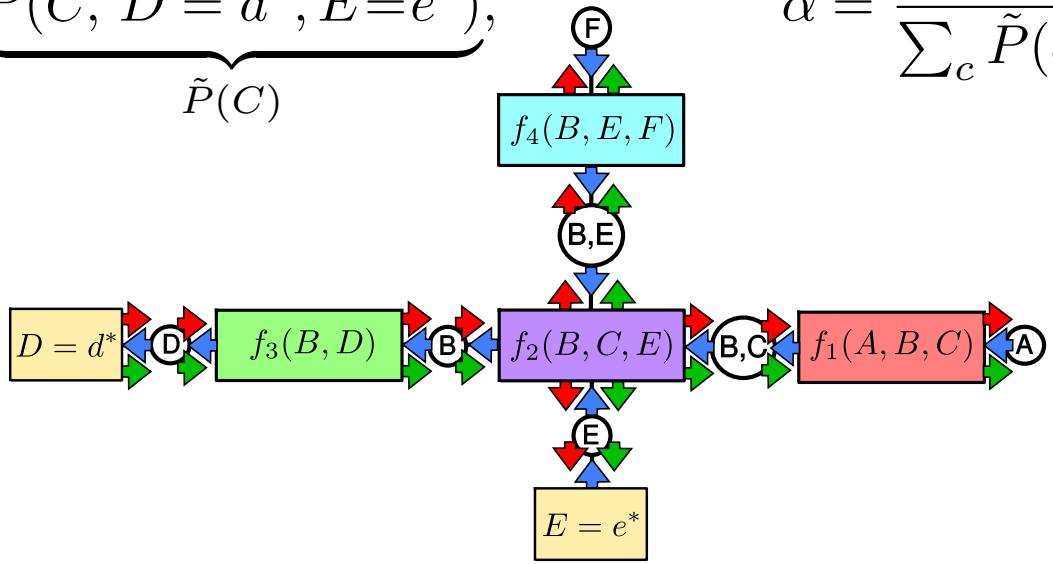


Inference

1. initialization of the clique potentials $f_k(\mathcal{X}_k)$
2. modification of the clique potentials by the observed evidence
3. message passing
4. calculate marginals from messages
5. normalization yields conditional marginals

$$P(C\,|D = d^*, E{=}e^*) = \alpha\, \underbrace{P(C,\, D = d^*, E{=}e^*)}_{\tilde{P}(C)}, \qquad \alpha = \frac{1}{\sum_c \tilde{P}(c)}$$





References I

C.M. Bishop. *Pattern Recognition and Machine Learning*. Springer Verlag, 2006. ISBN 978-0-387-31073-2.

Robert G Cowell, A Philip Dawid, Steffen L Lauritzen, and David J Spiegelhalter. Building and using probabilistic networks. *Probabilistic Networks and Expert Systems*, pages 25–41, 1999.

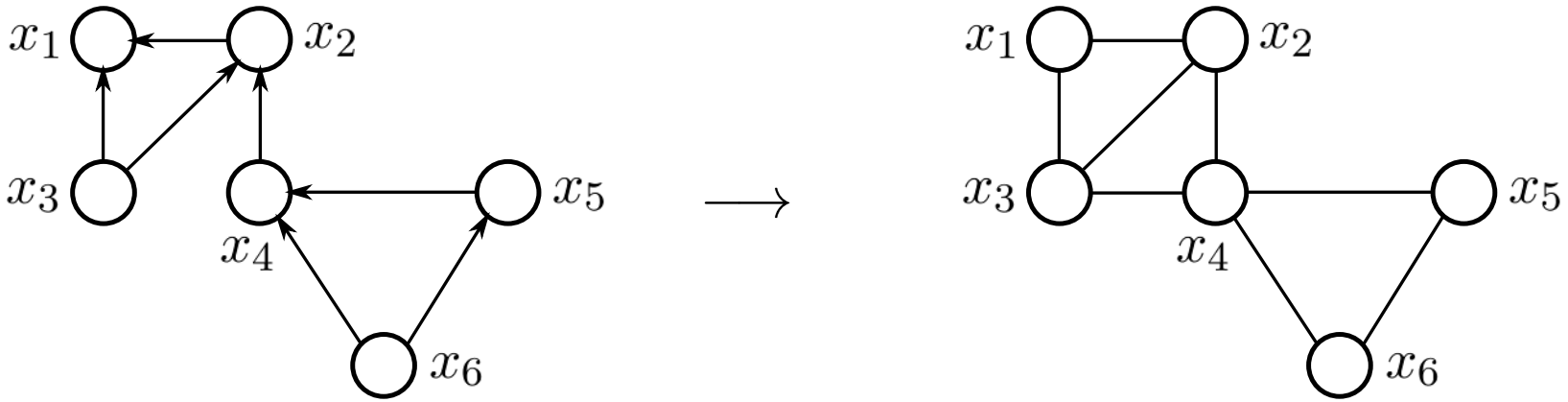
End of Section 3.2

the following slides contain

OPTIONAL MATERIAL

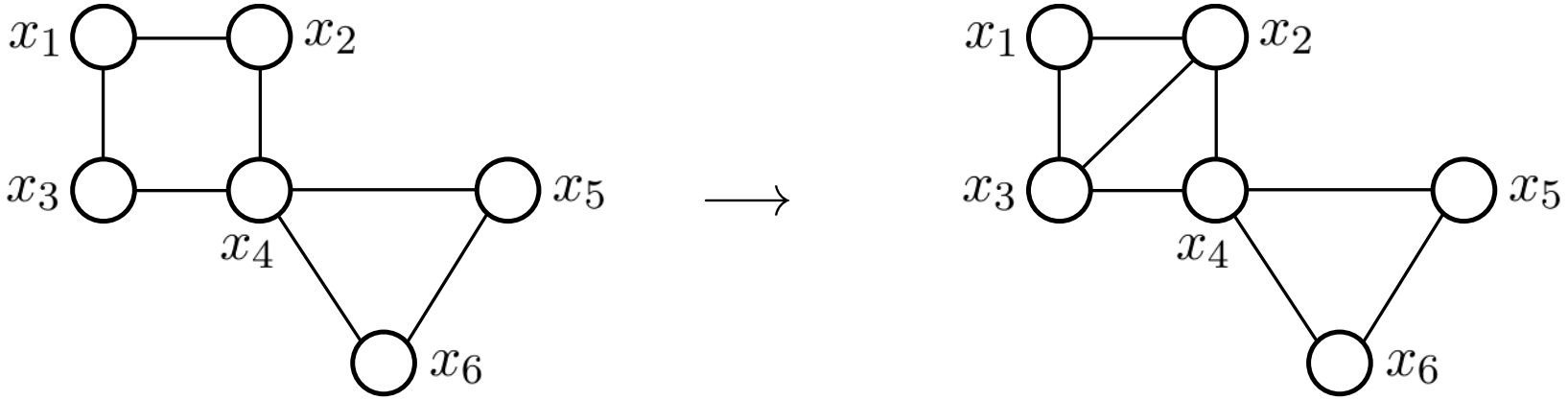
Moral graph

- **moral graphs** are the undirected equivalent of DAGs
  - connect nodes x_i of a DAG to their Markov blanket
  - insert undirected edges to all parents of all children of x_i
  - convert all directed to undirected edges



Chordal graph

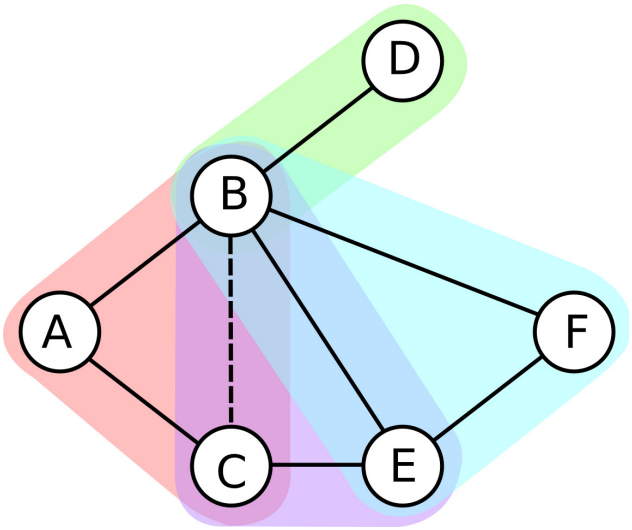
- **chordal graphs** become decomposable by inserting edges
  - insert undirected edges to all circles with 4 or more nodes



- transform DAG into chordal graph $\rightarrow$ junction tree

Decomposable graphs and distributions

- $A \perp B \,|\, C \Leftrightarrow A, B, C$ is proper decomposition of G
- decomposable graph factors into marginal distributions



$$P(\underline{\mathbf{x}}) = \frac{\prod_{\text{cliques } C} P_C(\underline{\mathbf{x}}_C)}{\prod_{\text{separators } S} P_S(\underline{\mathbf{x}}_S)}$$

$$P(A,B,C,D,E,F) = \frac{\overbrace{P(A,B,C)\,P(B,D)\,P(B,C,E)\,P(B,E,F)}^{\text{cliques}}}{\underbrace{P(B)\,P(B,C)\,P(B,E)}_{\text{separators}}}$$